FSMELD: Exact Subsequence Search under Elastic Measures via Amortized Sublinear-Time Lower-Bound Evaluation

Zemin Chao
Harbin Institute of Technology
Harbin, Heilongjiang, China
chaozm@hit.edu.cn

Zitong Li
Harbin Institute of Technology
Harbin, Heilongjiang, China
25S103343@stu.hit.edu.cn

John Paparrizos
The Ohio State University
Columbus, Ohio, United States
john@paparrizos.org

Zhixin Qi
Harbin Institute of Technology
Harbin, Heilongjiang, China
qizhx@hit.edu.cn

Hongzhi Wang*
Harbin Institute of Technology
Harbin, Heilongjiang, China
wangzh@hit.edu.cn

ABSTRACT

Elastic similarity measures support robust time-series matching, and lower bounds can yield order-of-magnitude speedups in exact search by safely pruning candidate subsequences. Subsequence range search can involve many overlapping windows but few matches to the query, making fast rejection of dissimilar candidates important. However, many existing lower bounds require $O(m)$ time per candidate for a query of length m, despite the substantial overlap between consecutive subsequences. Existing methods for amortized sublinear lower-bound evaluation are predominantly tailored to Dynamic Time Warping (DTW). We introduce FSMELD, the first unified framework for exact subsequence range search under *diverse* elastic measures, built on a generalized admissible lower bound (LB_{IP}). LB_{IP} expresses lower-bound contributions as products of binary masks derived from target envelopes and measure-specific query penalties. This decomposition enables joint evaluation across overlapping candidates through sliding inner products computed using the Fast Fourier Transform (FFT). We also preserve admissibility and shared evaluation under local z-normalization. Efficient FFT-based batch evaluation achieves an amortized lower-bound evaluation cost of $O(\log m)$ per candidate. The framework supports several elastic measures, including DTW, Edit Distance with Real Penalty (ERP), Edit Distance on Real Sequences (EDR), Longest Common Subsequence (LCSS), and Move-Split-Merge (MSM). Experiments on six datasets under ERP, EDR, LCSS, and MSM demonstrate average speedups of 46.1× in lower-bound evaluation and 2.4× in end-to-end exact search over the corresponding state-of-the-art baselines. Overall, FSMELD provides amortized sublinear lower-bound evaluation for multiple elastic measures, reducing repeated filtering work in exact subsequence similarity search.

PVLDB Reference Format:
Zemin Chao, Zitong Li, John Paparrizos, Zhixin Qi, and Hongzhi Wang. FSMELD: Exact Subsequence Search under Elastic Measures via Amortized Sublinear-Time Lower-Bound Evaluation. PVLDB, 20(1): XXX-XXX, 2027.
doi:XXX.xX

*Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XXX.xX

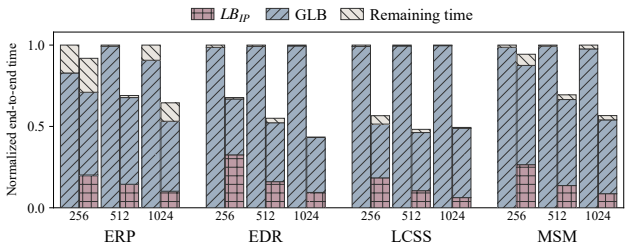


Figure 1: End-to-end runtime breakdown on RW for $m \in \{256, 512, 1024\}$. Each pair compares GLB+ELD (left) with FSMELD (right), both normalized by the corresponding GLB+ELD total runtime. Remaining time includes exact verification and other costs.

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/lzzt0123/FSMELD.

1 INTRODUCTION

Subsequence similarity search is a fundamental operation in time-series data management: given a query pattern, it identifies matching windows in a much longer sequence and supports downstream tasks such as retrieval, monitoring, motif discovery, and anomaly analysis [4, 11, 20]. A lock-step distance can change sharply when two otherwise similar patterns are locally shifted or stretched. Elastic measures address this problem by aligning observations in time, but the alignment also makes exact evaluation more expensive.

Dynamic Time Warping (DTW) is widely used, but comparative studies show that no single elastic measure is uniformly superior across the evaluated datasets [10, 27]. Lines and Bagnall also found that an elastic ensemble significantly outperformed its constituent classifiers in their benchmark [18]. In multivariate classification, Shifaz et al. found that all but one evaluated measure achieved the highest accuracy on at least one of 23 datasets [21]. Recent evaluations also highlight the value of non-DTW measures in clustering and multivariate analysis [9, 13]. This diversity is structural since different measures encode distinct matching semantics: Longest Common Subsequence (LCSS) tolerates unmatched observations [26]; Edit Distance on Real Sequences (EDR) uses tolerance-based edit costs [8]; Edit Distance with Real Penalty (ERP) uses real-valued matching and gap costs [7]; and Move-Split-Merge

(MSM) models move, split, and merge operations [22]. This diversity motivates a common acceleration framework for subsequence search that preserves each measure's matching semantics.

Motivation. Exact subsequence range search must identify all windows whose elastic distance to the query does not exceed a specified cutoff. For a query $Q \in \mathbb{R}^m$ and a target $S \in \mathbb{R}^n$, the search examines $n - m + 1$ overlapping candidate subsequences of length m. A dynamic program restricted to a width-w band typically costs $O(mw)$ per candidate, and local z-normalization must be applied consistently to every window when normalized matching is required. Exact systems therefore commonly use a filter–verify pipeline: a cheap admissible lower bound discards candidates whose bound exceeds the cutoff, and an exact evaluator processes the survivors. Research has improved both components. Strong linear-time pairwise bounds exist for DTW and have been generalized to other elastic measures, while early abandoning and state pruning reduce the work of exact dynamic programming [12, 19, 28].

Unlike whole-series matching, subsequence matching generates candidate windows at every valid starting position in a long target sequence. Occurrences of the query pattern are typically sparse, so most of these windows are dissimilar to the query and can be readily filtered out by lower bounds. Consequently, exact verification processes only a small fraction of the candidates, whereas filtering is applied across the entire candidate set. Although exact distance evaluation is more expensive per candidate, this disparity in candidate counts can make filtering more costly overall than verification. Figure 1 illustrates this bottleneck and the end-to-end savings from reducing filtering work. Yet evaluating a linear-time bound independently for each window repeats a length-m aggregation despite the overlap, motivating shared lower-bound computation. The Generalized Lower Bounding (GLB) framework supports multiple elastic measures and reuses per-series envelopes, but still requires $O(m)$ aggregation per candidate [19]. FSMDTW achieves batch-amortized $O(\log m)$ lower-bound evaluation per candidate for fixed-query DTW subsequence search [6]. However, amortized sublinear lower-bound evaluation across overlapping candidates remains an open challenge for multiple elastic measures.

Challenges. FSMDTW derives its sliding-inner-product formulation by expanding squared terms in a DTW-specific lower bound. Supporting ERP, EDR, LCSS, and MSM requires accounting for their fundamentally different cost semantics, including matching, gap, tolerance, and split–merge rules: a penalty that is unavoidable under DTW need not be unavoidable under another measure. The resulting envelope penalties involve clipping or tolerance-dependent tests that couple query values with target-envelope values, so the DTW-specific squared-cost expansion does not directly provide the required separable representation.

Moreover, local z-normalization can assign different normalized values to the same target observation across windows, complicating the reuse of target-side computations. The remaining challenge is thus to construct admissible, measure-specific lower-bound contributions that admit a common representation for shared evaluation across overlapping, locally normalized windows.

Contribution. To bridge this gap, we introduce FSMELD, the first unified framework for accelerating exact subsequence range search under DTW, ERP, EDR, LCSS, and MSM. Its core is LB_{IP}, an admissible lower-bound formulation whose contributions are products of target-side binary masks and measure-specific query penalties. We share upper and lower construction thresholds across candidate windows within each target segment to enable batch evaluation. For fixed thresholds, the target envelopes define binary masks indicating which positions satisfy the threshold conditions, while measure-specific query penalties determine the corresponding contribution magnitudes. We establish the admissibility of this construction, allowing the resulting bounds to safely prune candidates, and instantiate it for ERP, EDR, LCSS, and MSM. Table 1 compares FSMELD with state-of-the-art methods in terms of measure coverage, evaluation strategies, and per-candidate lower-bound costs.

Table 1: Comparison of measure coverage and lower-bound evaluation costs for length-m sliding candidates. Costs are per candidate and exclude exact verification. GLB costs assume available envelopes; it reuses per-series envelopes but aggregates each pair separately. The amortized costs use batches of length $\Theta(m)$ containing $\Theta(m)$ candidates; FSMELD includes heuristic threshold selection.

Method	Measure scope	Evaluation	Cost
GLB [19]	Multiple elastic measures	Pairwise aggregation	$O(m)$
FSMDTW [6]	DTW	Sliding inner products	$O(\log m)$
FSMELD (this paper)	DTW, ERP, EDR, LCSS, MSM	Shared thresholds and sliding inner products	$O(\log m)$

To support local z-normalization, we construct target envelopes valid across normalized candidate windows, preserving admissibility despite window-dependent normalization. Shared thresholds must also adapt to local data: selecting them too expensively can offset the savings from batch evaluation. We formulate shared-threshold selection as maximizing cumulative envelope penalties across candidates within a target segment. We derive an exact optimizer for this objective and an efficient heuristic that adapts the thresholds to local data without increasing the asymptotic batch-evaluation cost. For segments of length $\Theta(m)$ containing $\Theta(m)$ candidates, heuristic threshold selection and FFT-based evaluation of LB_{IP} together take amortized $O(\log m)$ time per candidate. Surviving candidates undergo further filtering and exact verification.

Results. We evaluate FSMELD on six datasets under ERP, EDR, LCSS, and MSM. The evaluation measures pruning effectiveness, lower-bound evaluation cost, and end-to-end exact range-search time, and includes cascade and threshold-selection ablations and parameter-sensitivity analyses. LB_{IP} achieves average lower-bound evaluation speedups of 46.1× over GLB and 12.0× over available measure-specific bounds. FSMELD achieves an average end-to-end search speedup of 2.40× over GLB+ELD. These results show that reducing repeated lower-bound computation translates into faster end-to-end exact search across the evaluated elastic measures. Our code is available at https://github.com/lzzt0123/FSMELD.

We first present the problem formulation and review related work (Section 2). The main contributions of this paper are:

- We propose LB_{IP}, the first unified inner-product lower-bound formulation with provable admissibility and instantiations for ERP, EDR, LCSS, and MSM (Section 3).

- We develop an exact optimizer and an efficient heuristic for shared-threshold selection over target segments. We analyze the threshold dependence of LB_{IP} to establish a finite search space and derive thresholds that maximize the bound for individual subsequence pairs (Section 4).
- We develop FSMELD for exact subsequence range search, supporting local z-normalization and amortized $O(\log m)$ evaluation of LB_{IP} per candidate (Section 5).
- Experiments under four elastic measures demonstrate high pruning rates and end-to-end speedups over state-of-the-art exact search pipelines (Section 6).

Finally, Section 7 summarizes our findings and concludes the paper.

2 PRELIMINARIES AND RELATED WORK

In this section, we first present the core definitions and problem formulation (Section 2.1), then introduce the elastic similarity measures and their dynamic programming formulations (Section 2.2), and finally review existing lower bounds and search acceleration techniques for elastic measures (Section 2.3).

2.1 Definitions and Problem Formulation

Definition 1 (Time Series). *A time series $S \in \mathbb{R}^n$ of length n is an ordered sequence of n real values, denoted $S = (S[1], S[2], \ldots, S[n])$, or equivalently $S = (s_1, s_2, \ldots, s_n)$ when subscript notation is used.*

Definition 2 (Subsequence). *A subsequence of a time series $S \in \mathbb{R}^n$ is a contiguous segment within S. We denote the subsequence starting at the i-th position as $S_{(i)} = (S[i], S[i+1], \ldots, S[i+m-1])$, where m is the length of the subsequence.*

Definition 3 (Upper and Lower Envelopes). *Given a time series segment $T \in \mathbb{R}^\ell$ and a temporal constraint window $w \in \mathbb{N}^+$, the upper envelope $\mathbb{U}^T \in \mathbb{R}^\ell$ and lower envelope $\mathbb{L}^T \in \mathbb{R}^\ell$ bound the local extrema of T within radius w:*

$$\mathbb{U}_i^T = \max_{\substack{1 \le j \le \ell \\ |i-j| \le w}} T[j], \qquad \mathbb{L}_i^T = \min_{\substack{1 \le j \le \ell \\ |i-j| \le w}} T[j]. \tag{1}$$

Throughout this paper, $ELD(X, Q)$ denotes the exact elastic distance under the temporal constraint w shared by all envelopes and lower bounds. We omit w from its notation.

Definition 4 (Subsequence Matching Problem under Elastic Similarity Measures). *Given a target time series $S \in \mathbb{R}^n$, a query $Q \in \mathbb{R}^m$ with $m \le n$, an elastic measure $ELD(\cdot, \cdot)$, and a cutoff $\xi \ge 0$, subsequence matching returns the starting indices*

$$\mathcal{R}_\xi(S, Q) = \left\{ i \in \{1, \ldots, n-m+1\} : ELD\Big(S_{(i)}, Q\Big) \le \xi \right\}. \tag{2}$$

We assume a fixed cutoff for ease of presentation; FSMELD applies to every $\xi \ge 0$.

2.2 Elastic Similarity Measures

Elastic similarity measures address temporal misalignment in real-world time series by allowing one-to-many or many-to-many point alignments. Table 2 summarizes the movement cost functions and final transformations for widely used elastic measures.

DTW is the best-known elastic measure [2], using dynamic programming to find an optimal alignment path that minimizes cumulative distance. However, empirical evidence indicates that DTW is highly sensitive to noise and outliers. To improve robustness, EDR [8] and LCSS [26] introduce a matching tolerance threshold ϵ. EDR assigns a unit edit penalty when two aligned elements differ by more than ϵ, whereas LCSS treats elements within ϵ as a match and maximizes the number of matched pairs. We convert the resulting LCSS similarity into a non-negative normalized distance, so that all elastic similarity measures considered in this paper share the same minimization objective. Throughout this paper, *RAC*, *ELD*, all envelopes, and all lower bounds use the same temporal constraint w. For a selected elastic measure, let $RAC(i, j)$ (Raw Accumulated Cost) denote the minimum accumulated cost between prefixes $X[1:i]$ and $Q[1:j]$, with $RAC(0, 0) = 0$.

For DTW, EDR, ERP, and MSM, states outside the band are unreachable, i.e., $RAC(i, j) = +\infty$ for $|i - j| > w$. For every reachable interior state,

$$\begin{aligned} RAC(i, j) = \min \big\{ & RAC(i-1, j-1) + \mathbf{M}(x_i, q_j), \\ & RAC(i-1, j) + \mathbf{D}_V(x_i, q_j), \\ & RAC(i, j-1) + \mathbf{D}_H(x_i, q_j) \big\}. \end{aligned} \tag{3}$$

Boundary initialization is measure-specific. DTW and MSM use $RAC(i, 0) = RAC(0, j) = +\infty$ for $i, j > 0$. Inside the band, EDR uses $RAC(i, 0) = i$ and $RAC(0, j) = j$, whereas the squared-cost ERP uses $RAC(i, 0) = \sum_{k=1}^{i} (x_k - g)^2$ and $RAC(0, j) = \sum_{k=1}^{j} (q_k - g)^2$; their remaining boundary states are $+\infty$. The resulting distance is

$$ELD(X, Q) = \mathrm{trans}(RAC(n, m)). \tag{4}$$

LCSS is represented in the same RAC framework by minimizing the number of unmatched query elements. Its temporal constraint applies only to admissible matches. Let $\delta(i, j) = 0$ if $|x_i - q_j| \le \epsilon$ and $|i - j| \le w$, and $\delta(i, j) = 1$ otherwise. With $RAC(i, 0) = 0$ and $RAC(0, j) = j$,

$$\begin{aligned} RAC(i, j) = \min\{ & RAC(i-1, j),\ RAC(i, j-1) + 1, \\ & RAC(i-1, j-1) + \delta(i, j) \}. \end{aligned} \tag{5}$$

For the equal-length pairs considered in this paper,

$$d_{\mathrm{LCSS}}(X, Q) = \frac{RAC(m, m)}{m} = 1 - \frac{\mathrm{LCSS}(X, Q)}{m}. \tag{6}$$

Consequently, a smaller value always indicates greater similarity between the compared sequences, consistent with the minimization objective in Definition 4. Despite their diverse mathematical formulations and proven effectiveness, these dynamic-programming-based measures suffer from quadratic worst-case time complexity [3, 4], limiting their direct use on massive time series datasets.

2.3 Existing Lower Bounds and Search Acceleration

Exact elastic-distance search commonly follows a filter–verify pipeline: an admissible lower bound prunes candidates with no false dismissals, and exact dynamic programming verifies the survivors. NeuTS supports approximate top-k search for elastic measures [29], but its learned score has not been established as an admissible lower

[1] For equal-length pairs, LCSS is expressed as a non-negative loss-minimization RAC with $RAC(i, 0) = 0$ and $RAC(0, j) = j$. Hence, $RAC(m, m) = m - \mathrm{LCSS}(X, Q)$ and $d_{\mathrm{LCSS}}(X, Q) = RAC(m, m)/m$. The temporal constraint applies only to admissible LCSS matches.

Table 2: Movement costs and final transformations for five elastic measures.

	DTW	LCSS[1]	EDR	ERP	MSM
$\mathbf{M}(x_i, q_j)$	$(x_i - q_j)^2$	$\begin{cases} 0, & \lvert x_i - q_j \rvert \le \epsilon \text{ and } \lvert i-j \rvert \le w, \\ 1, & \text{otherwise} \end{cases}$	$\begin{cases} 0, & \lvert x_i - q_j \rvert \le \epsilon, \\ 1, & \text{otherwise} \end{cases}$	$(x_i - q_j)^2$	$\lvert x_i - q_j \rvert$
$\mathbf{D_V}(x_i, q_j)$	$(x_i - q_j)^2$	0	1	$(x_i - g)^2$	$\begin{cases} c, & x_{i-1} \le x_i \le q_j \text{ or } x_{i-1} \ge x_i \ge q_j, \\ c + \min\{\lvert x_i - x_{i-1} \rvert,\ \lvert x_i - q_j \rvert\}, & \text{otherwise} \end{cases}$
$\mathbf{D_H}(x_i, q_j)$	$(x_i - q_j)^2$	1	1	$(q_j - g)^2$	$\begin{cases} c, & q_{j-1} \le q_j \le x_i \text{ or } q_{j-1} \ge q_j \ge x_i, \\ c + \min\{\lvert q_j - q_{j-1} \rvert,\ \lvert q_j - x_i \rvert\}, & \text{otherwise} \end{cases}$
$\mathrm{trans}(\cdot)$	$\sqrt{RAC(n,m)}$	$RAC(m,m)/m$	$RAC(n,m)$	$\sqrt{RAC(n,m)}$	$RAC(n,m)$

bound for safe pruning. Such approximate retrieval methods are therefore beyond the scope of this paper.

2.3.1 Measure-Specific Lower Bounds. DTW has the richest body of lower-bound research. Early bounds such as LB_{Yi} and LB_{Kim} use global features, whereas LB_{Keogh} compares one series to the warping envelope of the other [15, 16, 31]. Subsequent linear-time bounds, including LB_{Improved}, LB_{Enhanced}, and LB_{Webb}, improve the tightness–cost trade-off [17, 23, 28]. UCR Suite integrates cascading bounds, reordered evaluation, and early abandoning for exact DTW subsequence search [20]. EAPruned further accelerates DTW subsequence search in UCR Suite by integrating pruning and early abandoning within dynamic programming [12]. SWAMP uses a multiresolution hierarchy of admissible DTW lower bounds for exact motif discovery [1]. Beyond DTW, measure-specific bounds have been developed for the original ERP formulation [7] and LCSS [25, 26]. FastEE introduced bounds for MSM and TWED [24]. The original EDR study proposed three exact pruning filters [8].

2.3.2 Generalized Lower Bounding. Paparrizos et al. [19] proposed the Generalized Lower Bounding (GLB) framework for multiple elastic measures, including the first reported general lower bound for EDR. After envelope preprocessing, GLB still requires $O(m)$ pairwise aggregation for length-m sequences. GLB+ELD serves as our direct general-purpose filter–verify baseline.

2.3.3 Batch and Amortized Evaluation. FFT-based variants of MASS compute z-normalized Euclidean distance profiles through sliding inner products, including evaluation over overlapping blocks [32]. DTW-specific work also exploits reuse across related comparisons. An amortized $O(1)$ lower bound has been developed for motif discovery [5], while FSMDTW achieves batch-amortized $O(\log m)$ lower-bound evaluation per sliding candidate in subsequence search [6]. FSMELD formulates admissible lower bounds for ERP, EDR, LCSS, and MSM as FFT-compatible sliding inner products.

3 UNIFIED INNER-PRODUCT LOWER BOUND

In this section, we present LB_{IP}, the unified inner-product lower-bound framework underlying FSMELD. The key idea is to construct admissible envelope penalties as products of target-dependent binary masks and measure-specific query penalties. Sharing thresholds across overlapping candidates turns their lower-bound evaluation into sliding inner products that can be computed in batches using FFT. We first formulate LB_{IP} (Section 3.1) and prove its lower-bounding property (Section 3.2). We then explain the mechanism using a "water level" metaphor (Section 3.3) and show how the formulation enables amortized sublinear-time lower-bound evaluation (Section 3.4). Finally, we instantiate the proposed framework for ERP, EDR, LCSS, and MSM (Section 3.5).

3.1 Formulation of the LB_{IP} Framework

For conciseness, we denote the indicator function of predicate P as:

$$\mathbf{1}_{\{P\}} = \begin{cases} 1 & \text{if } P \text{ is true} \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

The proposed LB_{IP} framework constructs a lower bound by independently evaluating the dissimilarity contributions from the upper and lower envelopes of the target sequence.

Definition 5. *Given time series $X, Q \in \mathbb{R}^m$ and a cost-based elastic measure $ELD(X, Q)$, suppose its envelope lower-bound contribution can be expressed through a matching cost M and a deletion (or gap) penalty D (defined below). We define the inner-product-based lower bound LB_{IP} for any thresholds $t_U, t_L \in \mathbb{R}$ as:*

$$LB_{IP}(X, Q, t_U, t_L) = \mathrm{trans}\left(\sum_{i=1}^{m} g_U(i, t_U) \cdot h_U(i, t_U) + \sum_{i=1}^{m} g_L(i, t_L) \cdot h_L(i, t_L) \right) \tag{8}$$

where the valley mask functions g_U and g_L are defined as:

$$g_U(i,t) = \mathbf{1}_{\{t \ge \mathbb{U}_i^X\}}, \quad g_L(i,t) = \mathbf{1}_{\{t \le \mathbb{L}_i^X\}} \tag{9}$$

For cost-based formulations, the protrusion penalty functions h_U and h_L are defined as:

$$h_U(i,t) = \begin{cases} \min\{M(q_i,t), D(q_i)\}, & t < q_i \\ 0 & \textit{otherwise} \end{cases} \tag{10}$$

$$h_L(i,t) = \begin{cases} \min\{M(q_i,t), D(q_i)\}, & q_i < t \\ 0 & \textit{otherwise} \end{cases} \tag{11}$$

In this formulation, $M(\cdot)$ represents the distance-dependent matching cost function, and $D(\cdot)$ denotes the corresponding deletion or gap penalty for the cost-based measures. LCSS does not use this M–D interpretation; instead, its protrusion penalty function is instantiated separately as a binary function in Section 3.5.

3.2 The Lower Bound Property of LB_{IP}

The validity of the FSMELD framework requires that LB_{IP} consistently underestimate the true elastic distance, regardless of the threshold parameters, as formalized in the following theorem.

Theorem 1. *Given time series $X \in \mathbb{R}^m$ and $Q \in \mathbb{R}^m$, for any cost-based elastic measure $ELD(X, Q)$ satisfying Definition 5, and any given thresholds $t_U, t_L \in \mathbb{R}$:*

$$ELD(X, Q) \geq LB_{IP}(X, Q, t_U, t_L) \tag{12}$$

Proof. A lower bound for $ELD(X, Q)$ can be constructed from two symmetric components: Γ_U, representing the dissimilarity contribution between the upper envelope $\mathbb{U}^X$ and Q, and Γ_L, representing the contribution between the lower envelope $\mathbb{L}^X$ and Q. Therefore,

$$RAC(X, Q) \geq \Gamma_U + \Gamma_L \tag{13}$$

where the final distance is obtained by applying a monotonically non-decreasing transformation function $\text{trans}(\cdot)$ to the non-negative raw accumulated cost. Following Paparrizos et al. [19], Γ_U can be lower-bounded as:

$$\Gamma_U \geq \sum_{i=1}^{m} \begin{cases} \min\{M(q_i, \mathbb{U}_i^X), D(q_i)\}, & \mathbb{U}_i^X < q_i \\ 0, & \text{otherwise} \end{cases} \tag{14}$$

Thus, we derive a weaker but valid lower bound by introducing a threshold $t_U \in \mathbb{R}$ and restricting attention to regions where the envelope lies at or below this threshold (i.e., $\mathbb{U}_i^X \leq t_U$). For the cost-based measures considered, $M(q_i, t)$ is non-negative and non-increasing in t on the active interval $\mathbb{U}_i^X \leq t < q_i$. Therefore, whenever $\mathbb{U}_i^X \leq t_U < q_i$, we have $M(q_i, \mathbb{U}_i^X) \geq M(q_i, t_U)$. Outside this interval, $g_U(i, t_U)h_U(i, t_U) = 0$. Encoding this condition as the indicator mask $g_U(i, t_U) = \mathbf{1}_{\{t_U \geq \mathbb{U}_i^X\}}$ and substituting the penalty $h_U(i, t_U)$ yields the following inequality:

$$\begin{aligned} \Gamma_U &\geq \sum_{i=1}^{m} \begin{cases} \min\{M(q_i, t_U), D(q_i)\}, & \mathbb{U}_i^X \leq t_U < q_i \\ 0, & \text{otherwise} \end{cases} \\ &= \sum_{i=1}^{m} \mathbf{1}_{\{t_U \geq \mathbb{U}_i^X\}} \times \begin{cases} \min\{M(q_i, t_U), D(q_i)\}, & t_U < q_i \\ 0, & \text{otherwise} \end{cases} \\ &= \sum_{i=1}^{m} g_U(i, t_U) \times h_U(i, t_U) \end{aligned} \tag{15}$$

By symmetry, applying identical logic to the lower envelope yields $\Gamma_L \geq \sum_{i=1}^{m} g_L(i, t_L) \cdot h_L(i, t_L)$. Summing these two independent components directly yields:

$$RAC(X, Q) \geq \sum_{i=1}^{m} g_U(i, t_U) \cdot h_U(i, t_U) + \sum_{i=1}^{m} g_L(i, t_L) \cdot h_L(i, t_L) \tag{16}$$

Because the transformation function $\text{trans}(\cdot)$ associated with the elastic measure ELD is monotonically non-decreasing, applying it to both sides preserves the inequality. Thus, we establish that:

$$\begin{aligned} ELD(X, Q) &\geq \text{trans}\left(\sum_{i=1}^{m} g_U \cdot h_U + \sum_{i=1}^{m} g_L \cdot h_L \right) \\ &= LB_{IP}(X, Q, t_U, t_L) \end{aligned} \tag{17}$$

which completes the proof. □

3.3 Intuitive Explanation of LB_{IP}

To provide an intuitive understanding of the mathematical formulation, we use a "water level and penalty filtering" metaphor as illustrated in Figure 2. The threshold t_U acts as a global "water level" that decouples the target series' morphology from the query series' amplitude. Specifically, $g_U(i, t_U)$ serves as a binary mask identifying the "valleys" within the target sequence. If the upper envelope $\mathbb{U}_i^X$ falls at or below the water level t_U, the position marks a region in which a penalty may be incurred (yielding $g_U = 1$). Otherwise, the mask is 0 and the target position is inactive.

On the query side, $h_U(i, t_U)$ quantifies the inherent "protrusion penalty" of Q. If a query element q_i acts as a protrusion rising above t_U, forcing its alignment with any submerged target element incurs an unavoidable baseline cost. The function h_U captures this minimum guaranteed penalty, which represents either the cost of matching down to the water level $M(q_i, t_U)$ or the fixed deletion cost $D(q_i)$. Consequently, the framework transforms the lower-bound computation for complex elastic alignments into a decoupled sliding inner product. A valid penalty is accumulated only when a protruding query element ($h_U > 0$) aligns with a guaranteed target valley ($g_U = 1$). This decoupling enables amortized sublinear-time evaluation through the Convolution Theorem.

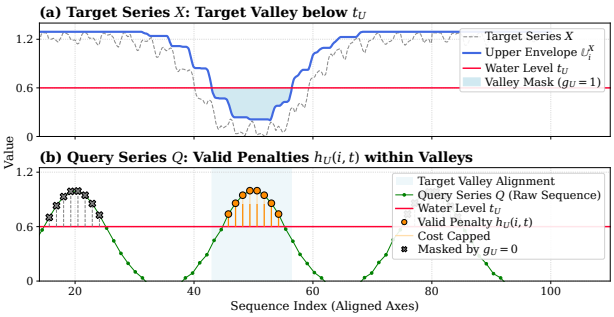


Figure 2: Water-level interpretation of the upper-envelope penalty contribution in LB_{IP} (denoted by $\Delta_U(t_U)$ in Section 4). (a) t_U activates target valleys through g_U. (b) Only query protrusions aligned with active valleys contribute to the bound.

3.4 Amortized Sublinear–Time LB_{IP} Evaluation

In subsequence matching, LB_{IP} must be evaluated for every length-m candidate subsequence within a target segment $T \in \mathbb{R}^\ell$ with $\ell > m$. Let $k \in \{1, 2, \ldots, \ell - m + 1\}$ denote the starting index of a candidate window. We use the same thresholds t_U, t_L across all candidates in the segment. For these fixed thresholds, the target masks g_U, g_L are constructed from the segment's envelopes, while the measure-specific penalties h_U, h_L depend only on the query Q. This separation allows lower bounds based on these shared envelopes to be evaluated in batches through sliding inner products. We construct binary target mask vectors $A, A' \in \{0, 1\}^\ell$ and query penalty vectors $B, B' \in \mathbb{R}^m$ as follows:

$$A = (g_U(1, t_U), g_U(2, t_U), \ldots, g_U(\ell, t_U)) \tag{18}$$

$$B = (h_U(1, t_U), h_U(2, t_U), \ldots, h_U(m, t_U)) \tag{19}$$

$$A' = (g_L(1, t_L), g_L(2, t_L), \ldots, g_L(\ell, t_L)) \tag{20}$$

$$B' = (h_L(1, t_L), h_L(2, t_L), \ldots, h_L(m, t_L)) \tag{21}$$

Let $A_{(k)}$ and $A'_{(k)}$ denote the contiguous length-m subvectors of A and A' starting at index k, which represent the active masks for the k-th sliding window. Let $\langle \cdot, \cdot \rangle$ denote the standard vector inner product. The expression of LB_{IP} for the k-th candidate subsequence $T_{(k)}$ can be transformed from its piecewise sum into two vector inner products:

$$LB_{IP}(T_{(k)}, Q, t_U, t_L) = \text{trans}\left(\langle A_{(k)}, B \rangle + \langle A'_{(k)}, B' \rangle\right) \tag{22}$$

By recasting the alignment into vectors, evaluating the lower bounds across all $N_w = \ell - m + 1$ subsequences becomes mathematically equivalent to a discrete linear convolution between the target mask array and the reversed query penalty array. By the Convolution Theorem, these convolutions can be computed with the FFT-based Fast Sliding Inner Product routine (SIP, Algorithm 1), giving a total lower-bound evaluation cost of $O(\ell \log \ell)$ for all $N_w = \ell - m + 1$ candidates. Following established practices [6, 30], we use segments of length $\ell = \Theta(m)$ containing $N_w = \ell - m + 1 = \Theta(m)$ candidates. The $O(\ell \log \ell)$ batch cost then yields an amortized lower-bound evaluation cost of $O(\log m)$ per candidate.

Algorithm 1: SIP(T, X)

Input: $T \in \mathbb{R}^{\ell}$, $X \in \mathbb{R}^{m}$, $(\ell > m)$.
Output: **The inner product between** X **and** $T_{(i)}$ **for** $i \in \{1, 2, \ldots, \ell - m + 1\}$.

$X_{pad} \leftarrow$ reverse X and pad with $\ell - m$ trailing zeros;
$COV \leftarrow IDFT\left(DFT(T) \odot DFT(X_{pad})\right)$; // $O(\ell \log \ell)$
return the real part of $COV[m - 1 : \ell]$.

3.5 Instantiating LB_{IP} Across Elastic Measures

To demonstrate the generality of the LB_{IP} framework, we instantiate it for four widely used yet computationally expensive elastic measures: ERP, EDR, LCSS, and MSM. The target-side valley mask functions $g_U(i, t)$ and $g_L(i, t)$ retain the same form across elastic measures. Instantiating FSMELD for a specific measure requires specifying the query-side protrusion penalty functions $h_U(i, t)$ and $h_L(i, t)$, together with the transformation trans$(\cdot)$ from accumulated penalties to the final lower bound LB_{IP}. Table 3 summarizes the metric-specific upper-side penalties $h_U(i, t)$ for ERP, EDR, LCSS, and MSM. For EDR and LCSS, the binary protrusion penalties incorporate the matching tolerance ϵ. For MSM, the protrusion penalty is capped at c, the base split/merge cost parameter.

LCSS-specific instantiation. For LCSS, the binary penalties in Table 3 certify query positions that cannot participate in any admissible match; they do not represent edit costs. The upper and lower contributions cannot certify the same query position, so their sum counts distinct unmatched query positions. For equal-length pairs, trans$(z) = z/m$, yielding

$$LB_{IP}(X, Q, t_U, t_L) \le d_{\text{LCSS}}(X, Q). \tag{23}$$

The proof is given in Appendix A of our extended technical report.[2]

Table 3: Metric-specific upper-side penalties for FSMELD when $t < q_i$; otherwise, $h_U(i, t) = 0$.

Metric	Penalty $h_U(i, t)$
ERP	$\min\{(q_i - t)^2, (q_i - g)^2\}$
EDR (ϵ)	$\mathbf{1}_{\{q_i > t+\epsilon\}}$
LCSS (ϵ)	$\mathbf{1}_{\{q_i > t+\epsilon\}}$
MSM (c)	$\min\{c, q_i - t\}$

Transforming the raw time series into the required mask and penalty functions incurs at most amortized $O(1)$ time per element. This ensures that metric-specific preprocessing preserves the amortized sublinear cost of lower-bound evaluation.

4 THRESHOLD OPTIMIZATION FOR THE UNIFIED LOWER BOUND

This section presents algorithms for selecting shared thresholds (t_U, t_L) to improve the tightness and pruning effectiveness of LB_{IP} across a target segment. Although LB_{IP} is admissible for any thresholds, optimizing them incurs additional overhead beyond its amortized $O(\log m)$ evaluation cost for fixed thresholds. We first formulate the optimization objective (Section 4.1) and prove that an optimal threshold can always be selected from the finite set of target envelope values (Section 4.2). We then present an exact optimization algorithm (Section 4.3), followed by an efficient heuristic (Section 4.4). The heuristic retains an amortized $O(\log m)$ cost per candidate for threshold selection and lower-bound evaluation.

4.1 Objective Formulation

Given a target segment $T \in \mathbb{R}^{\ell}$ and a query sequence Q, we aim to select shared thresholds (t_U, t_L) that optimize the pruning effectiveness of the proposed LB_{IP} framework. To this end, a natural and effective optimization objective is to directly maximize the cumulative penalty contributions across all overlapping windows.

Definition 6 (Threshold Optimization Objective). *Let $k \in [1, \ell - m + 1]$ denote the sliding window offset. The optimal thresholds t_U^* and t_L^* independently maximize the cumulative penalty components across all windows:*

$$\begin{aligned} t_U^* &\in \arg\max_{t \in \mathbb{R}} \sum_{k=1}^{\ell-m+1} \Delta_U(T_{(k)}, Q, t), \\ t_L^* &\in \arg\max_{t \in \mathbb{R}} \sum_{k=1}^{\ell-m+1} \Delta_L(T_{(k)}, Q, t), \end{aligned} \tag{24}$$

where Δ_U and Δ_L represent the penalty contributions from the upper and lower envelopes for the subsequence $T_{(k)}$, computed using the shared segment envelopes $\mathbb{U}^T$ and $\mathbb{L}^T$, respectively.

[2]The extended version containing all appendices and additional proofs is available at https://github.com/lzzt0123/FSMELD/blob/main/FSMELD_Extended.pdf

The cumulative objective sums the upper- and lower-envelope contributions $\Delta_U(T_{(k)}, Q, t_U)$ and $\Delta_L(T_{(k)}, Q, t_L)$ over all candidate windows. Since these contributions depend on separate thresholds, the joint optimization decomposes into two independent and symmetric subproblems. For conciseness, the remainder of this section focuses on optimizing the upper threshold t_U; the lower threshold t_L is optimized symmetrically.[3]

4.2 Properties of the Objective Function

The following theorem reduces the continuous threshold search space to a discrete set of candidate values.

Theorem 2 (Discrete Search Space for Optimal Threshold). *For a target segment T and query sequence Q, an optimal threshold t_U^* for the cumulative objective in Definition 6 can be selected such that*

$$t_U^* \in \{\mathbb{U}_i^T\}_{i=1}^{\ell}. \tag{25}$$

Proof Sketch. By definition, the upper penalty for a fixed window $T_{(k)}$ is evaluated as $\Delta_U(T_{(k)}, Q, t_U) = \sum_{i=1}^{m} g_U(i+k-1, t_U) h_U(i, t_U)$. The valley mask $g_U(i+k-1, t_U) = \mathbf{1}_{\{t_U \geq \mathbb{U}_{i+k-1}^T\}}$ acts as an indicator function, introducing stepwise discontinuities at the corresponding target envelope values. Figure 3 illustrates an individual contribution, denoted by $\delta_U^{(i)}(t) = g_U(i+k-1, t) h_U(i, t)$, with $q = q_i$ and the envelope marker corresponding to $\mathbb{U}_{i+k-1}^T$.

Within any interval between two consecutive envelope values, the set of activated target valleys remains fixed. Concurrently, for any active query element, the protrusion penalty $h_U(i, t_U)$ is monotonically non-increasing as the threshold t_U rises.

Because $\Delta_U(T_{(k)}, Q, t_U)$ is non-increasing within each such interval for every window, the same property holds for the cumulative sum over all windows. Moving a threshold to the left envelope endpoint cannot decrease this cumulative sum. Therefore, an optimal threshold t_U^* maximizing the objective can always be selected from the discrete envelope set. □

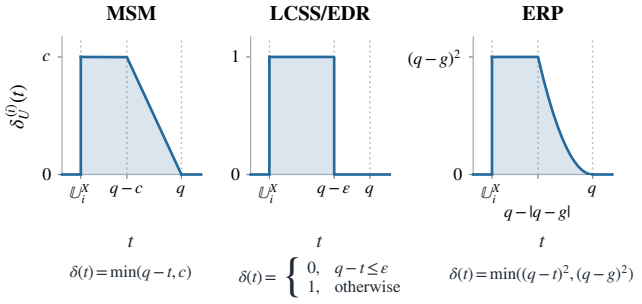


Figure 3: Individual upper-envelope contribution as a function of the threshold t. Let u denote the marked target-envelope value. The expressions below the plots apply for $u \leq t < q$; the contribution is zero otherwise.

[3]For an individual subsequence pair, the thresholds maximizing LB_{IP} can be computed in $O(m \log m)$ time (See Appendix B of the extended report).

4.3 Exact Threshold Optimization

Given a target segment $T \in \mathbb{R}^{\ell}$ and a query sequence $Q \in \mathbb{R}^m$, this subsection presents the method that computes the optimal threshold t_U^* in $O(m^2 + \ell \log \ell)$ time.

4.3.1 Objective Decomposition. By Theorem 2, the optimal upper threshold $t_U^* \in \{\mathbb{U}_i^T\}_{i=1}^{\ell}$. Assuming $\ell \geq 2m$, as in the segment configuration used by FSMELD, core target positions pair with every query position across the sliding windows, whereas boundary positions pair with only a subset. The core contribution can therefore be factored into a mask count and a total query penalty. Reindexing the cumulative objective by target position gives:

$$\begin{aligned} \sum_{k=1}^{\ell-m+1} \Delta_U(T_{(k)}, Q, t_U) &= \sum_{k=1}^{\ell-m+1} \sum_{i=1}^{m} g_U(i+k-1, t_U) \times h_U(i, t_U) \\ &= p_1(t_U) + p_2(t_U) + p_3(t_U) \end{aligned} \tag{26}$$

The core term $p_1(t_U)$ collects target positions that occur with every query position across the sliding windows:

$$p_1(t_U) = \left[\sum_{j=m}^{\ell-m+1} g_U(j, t_U) \right] \left[\sum_{i=1}^{m} h_U(i, t_U) \right]. \tag{27}$$

The boundary terms $p_2(t_U)$ and $p_3(t_U)$ capture the partial overlaps at the left and right edges of the segment:

$$p_2(t_U) = \sum_{j=1}^{m-1} g_U(j, t_U) \left[\sum_{i=\max\{1, j-\ell+m\}}^{j} h_U(i, t_U) \right], \tag{28}$$

$$p_3(t_U) = \sum_{j=\ell-m+2}^{\ell} g_U(j, t_U) \left[\sum_{i=j-(\ell-m)}^{\min\{m, j\}} h_U(i, t_U) \right]. \tag{29}$$

4.3.2 Exact Computation. This subsection presents the method that computes the optimal threshold t_U^* in $O(m^2 + \ell \log \ell)$ time. Appendix C provides the detailed pseudocode.

Theorem 3. *For a target segment T of length $\ell \geq 2m$ and a query Q of length m, the exact optimal threshold t_U^* for the objective in Definition 6 can be computed in $O(m^2 + \ell \log \ell)$ time.*

Proof Sketch. Each query penalty is piecewise polynomial (degree ≤ 2) with $O(1)$ breakpoints. Sorting the $O(\ell + m)$ envelope and query-penalty breakpoints requires $O(\ell \log \ell)$ time. During the sweep, the active core masks and aggregate polynomial coefficients of $\sum_i h_U(i, t)$ are incrementally maintained. Each transition updates these core statistics in $O(1)$ time, enabling $O(1)$ evaluation of $p_1(t)$.

Expanding $p_2(t)$ and $p_3(t)$ yields $O(m^2)$ products $g_U(j, t) h_U(i, t)$, whose polynomial coefficients are also aggregated. Each product is processed once upon mask activation and updated at most $O(1)$ times at the breakpoints of its associated query penalty. All updates use the already-sorted events, so maintaining the boundary terms requires $O(m^2)$ total work. At a shared event value, query transitions are processed before mask activations to respect the penalty values at the breakpoints. After these updates, the full objective is evaluated in $O(1)$ time using the maintained aggregates. By Theorem 2, checking all envelope thresholds guarantees an exact maximizer, yielding an overall $O(m^2 + \ell \log \ell)$ complexity. The lower-threshold computation follows symmetrically. □

Choosing $\ell = Km^2$ for a fixed $K \geq 2$ amortizes threshold optimization to $O(\log m)$ per candidate, but a single threshold may not suit all windows in such a long segment. Section 4.4 therefore applies the same incremental penalty maintenance to $p_1(t)$, supporting shorter segments with $\ell = \Theta(m)$.

4.4 Fast $O(\ell \log \ell)$ Time Threshold Optimization

4.4.1 Heuristic Optimization Objective. To reduce threshold selection overhead for segments of length $\ell = \Theta(m)$, we omit the boundary terms $p_2(t)$ and $p_3(t)$ and maximize $p_1(t)$ alone. The selected threshold maximizes the core objective and serves as a heuristic choice for the full cumulative penalty. The boundary terms are omitted only when selecting thresholds; subsequent LB_{IP} evaluation still uses all candidate positions. Since LB_{IP} is admissible for any thresholds, this choice preserves exact subsequence matching. The resulting cost–pruning trade-off is evaluated in Section 6.4. Applying the monotonicity argument in Section 4.2 to $p_1(t)$, a maximizer can be selected from the core envelope values. Let $\mathcal{U} \in \mathbb{R}^\ell$ be the upper-bounding envelope of a segment. Algorithm 2 extracts $\mathcal{U}_{core} = \mathcal{U}[m : \ell - m + 1]$ and sorts its distinct values to construct the candidate set $\mathcal{T}$. The frequency of each value is precomputed, allowing the activation count G_U, the first factor of $p_1(t)$, to be updated without repeatedly scanning the core envelope.

4.4.2 Dynamic State Tracking and Cost Maintenance. The second factor of $p_1(t)$ is the total query protrusion penalty $\Delta_Q(Q, t)$. The heaps track only query penalties; the target-side masks are counted separately by G_U. Let $\tau(q_i)$ denote the end of the fixed-penalty branch. As t increases, each q_i belongs to one of three states:

- **State 1 (Fixed Penalty):** When $t < \tau(q_i)$, the contribution is $D(q_i)$. We maintain its index in a min-heap H_2, ordered by $\tau(q_i)$.
- **State 2 (Post-transition Penalty):** When $\tau(q_i) \leq t < q_i$, the element follows the post-transition branch of $h_U(i, t)$: quadratic for ERP and DTW, linear for MSM, and zero for EDR and LCSS. We maintain its index in a min-heap H_1, ordered by q_i.
- **State 3 (Submerged):** When $t \geq q_i$, the element contributes zero and is absent from both heaps.

DTW has no fixed-penalty state; we use $\tau(q_i) = -\infty$ in the algorithm. For EDR and LCSS, the fixed branch of $h_U(i, t)$ has value 1. For LCSS, $D(q_i) = 1$ is used solely as a bookkeeping convention in this threshold-selection procedure: it represents a certified lost match, not an LCSS deletion cost. The zero-contribution entries of EDR and LCSS are retained in H_1 to use the same state-transition procedure for all measures. Table 4 lists the transition boundaries.

At any given threshold t, the total query protrusion penalty is

$$\begin{aligned} \Delta_Q(Q, t) &= \sum_{i=1}^{m} h_U(i, t) \\ &= \sum_{i \in H_1} h_U(i, t) + \sum_{i \in H_2} D(q_i). \end{aligned} \tag{30}$$

Here, summations over H_1 and H_2 refer to their stored query indices. We maintain the moments $\mathcal{M}_\alpha = \sum_{i \in H_1} q_i^\alpha$, where $\mathcal{M}_0 = |H_1|$. The contribution of H_1 can then be evaluated in constant time via the polynomial function f_p. For MSM,

$$\sum_{i \in H_1} h_U(i, t) = \mathcal{M}_1 - t|H_1|. \tag{31}$$

For ERP and DTW,

$$\sum_{i \in H_1} h_U(i, t) = \mathcal{M}_2 - 2t\mathcal{M}_1 + t^2|H_1|. \tag{32}$$

The right-hand sides define $f_1(\mathcal{M}_1, |H_1|, t)$ and $f_2(\mathcal{M}_2, \mathcal{M}_1, |H_1|, t)$, respectively. Here, p denotes the highest moment order required by the measure: $p = 2$ for ERP and DTW, and $p = 1$ for MSM. EDR and LCSS require only counts ($p = 0$), with zero contribution from H_1, i.e., $f_0(|H_1|, t) = 0$. For $p = 0$, the positive-order moment arguments are omitted. The fixed-penalty sum $\sigma_D = \sum_{i \in H_2} D(q_i)$ is maintained separately for the fixed-penalty state.

Table 4: State transition boundaries $\tau(q_i)$ for core elastic measures (assuming $q_i > t$).

Elastic Metric	State-transition condition	$\tau(q_i)$ (Condition of $H_2 \to H_1$)
DTW	no H_2	no condition
LCSS	$q_i - t = \epsilon$	$q_i - \epsilon$
EDR	$q_i - t = \epsilon$	$q_i - \epsilon$
ERP	$q_i - t = \lvert g - q_i \rvert$	$q_i - \lvert q_i - g \rvert$
MSM	$q_i - t = c$	$q_i - c$

4.4.3 Threshold Selection Algorithm and Complexity. Algorithm 2 initializes the query states at $t_0 = \mathcal{T}_1$. At each candidate threshold, it first transfers elements with $\tau(q_i) \leq t$ from H_2 to H_1, then removes elements with $q_i \leq t$ from H_1. After these updates, $\Delta_Q(Q, t) = \sum_i h_U(i, t)$ and G_U equals the number of core envelope values at most t. Their product therefore gives $p_1(t)$ exactly. Equality must be processed before evaluation, particularly for EDR and LCSS, whose penalties drop to zero at $t = q_i - \epsilon$. The best candidate in $\mathcal{T}$ therefore maximizes the heuristic objective.

Sorting the core envelope values and recording their frequencies requires $O(\ell \log \ell)$ time. Each query element enters and exits each heap at most once, giving $O(m \log m)$ heap maintenance. The count operation retrieves the corresponding precomputed frequency in $O(1)$ time. After state updates, evaluating each candidate threshold takes $O(1)$ time. Since $m \leq \ell$, the total threshold-selection cost is $O(\ell \log \ell)$. With $\ell = \Theta(m)$, this gives an amortized $O(\log m)$ cost per candidate subsequence, preserving the amortized sublinear lower-bound evaluation cost of FSMELD. The lower threshold is obtained by the symmetric procedure.

5 THE FSMELD SEARCH ALGORITHM

In this section, we integrate LB_{IP} and the heuristic for threshold selection into FSMELD, a complete algorithm for exact subsequence matching. Within its cascaded filter–verify pipeline, FFT-based lower-bound evaluation takes amortized $O(\log m)$ time per candidate and prunes unpromising candidates before expensive exact verification. Since subsequence matching typically requires normalization [14], we focus on z-normalized matching, where each length-m candidate is normalized to zero mean and unit variance.

Algorithm 2: OptUpperThreshold($\mathcal{U}, Q$)

Input: $\mathcal{U} \in \mathbb{R}^{\ell}$, $Q \in \mathbb{R}^{m}$, $(\ell \geq 2m)$.
Output: The threshold value for LB_{IP}.

$\mathcal{U}_{core} \leftarrow \mathcal{U}[m : \ell - m + 1]$;
$\mathcal{T} \leftarrow$ Extract unique elements from $\mathcal{U}_{core}$ and record their counts ;
Sort $\mathcal{T}$ in ascending order ;
$H_1, H_2 \leftarrow$ Initialize two empty min-heaps ;
$G_U \leftarrow 0$; $t_0 \leftarrow \mathcal{T}_1$; $\sigma_D \leftarrow 0$; $\mathcal{M}_0, \mathcal{M}_1, \ldots, \mathcal{M}_p \leftarrow 0$;
for $j = 1$ *to* m **do**
  if $q_j \leq t_0$ **then**
    continue ; // $j \notin H_1 \cup H_2$
  if $\tau(q_j) > t_0$ **then**
    $H_2.insert(\tau(q_j), j)$; // Tuple of (thresh, idx)
    $\sigma_D \leftarrow \sigma_D + D(q_j)$; // Contribution of H_2.
  else
    $H_1.insert(q_j, j)$; // Tuple of (thresh, idx)
    for $\alpha = 0$ *to* p **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha + (q_j)^\alpha$; // Contribution of H_1.
for $i = 1$ *to* $|\mathcal{T}|$ **do**
  $t \leftarrow \mathcal{T}_i$;
  $G_U \leftarrow G_U + \text{count}(\mathcal{U}_{core}, \mathcal{T}_i)$;
  while ***not*** $H_2.empty()$ ***and*** $H_2.top().thresh \leq t$ **do**
    $k \leftarrow H_2.top().\text{idx}$;
    $\sigma_D \leftarrow \sigma_D - D(q_k)$; // $H_2 \leftarrow H_2 \setminus \{k\}$
    $H_1.insert(q_k, k)$; // $H_1 \leftarrow H_1 \cup \{k\}$
    for $\alpha = 0$ *to* p **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha + (q_k)^\alpha$;
    $H_2.pop()$;
  while ***not*** $H_1.empty()$ ***and*** $H_1.top().thresh \leq t$ **do**
    $k \leftarrow H_1.top().\text{idx}$;
    for $\alpha = 0$ *to* p **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha - (q_k)^\alpha$; // $H_1 \leftarrow H_1 \setminus \{k\}$
    $H_1.pop()$;
  $\Delta_Q \leftarrow \sigma_D + f_p(\mathcal{M}_p, \mathcal{M}_{p-1}, \ldots, \mathcal{M}_1, |H_1|, t)$;
  $LB_i \leftarrow \Delta_Q \times G_U$;
$idx \leftarrow \arg\max_{1 \leq i \leq |\mathcal{T}|} LB_i$;
return $\mathcal{T}_{idx}$; // t_L can be symmetrically obtained.

We first construct conservative envelopes for normalized windows (Section 5.1) and then present the cascaded FSMELD pipeline (Section 5.2). The raw-data algorithm is given in Appendix D.

5.1 Conservative Envelopes under Z-Normalization

Consider a target segment $T \in \mathbb{R}^{\ell}$ and its length-m candidate windows. Z-score normalization is frequently required to achieve scale and baseline invariance in time series mining applications. For a subsequence X, its normalized form $\hat{X}$ is computed as $\hat{X}[i] = \frac{X[i]-\mu}{\sigma}$, where μ and σ are the mean and standard deviation of X, respectively.[4] However, adapting FSMELD to normalized sequences introduces a critical challenge: a single element $T[i]$ in the target series can assume different normalized values depending on the enclosing length-m sliding window. Consequently, the normalized local envelope values associated with this position vary across the overlapping candidate windows.

To guarantee the lower-bound property (Theorem 1 and the LCSS-specific bound in Section 3.5) under z-normalization, we must conservatively estimate the boundaries of the normalized envelopes. For any position i within a local segment of length ℓ, let $\mu_i^{\min}$ and $\mu_i^{\max}$ denote the minimum and maximum local means, and $\sigma_i^{\min}$ and $\sigma_i^{\max}$ the minimum and maximum local standard deviations, over all non-degenerate length-m subsequences containing i. Positions covered only by degenerate windows are handled separately. These localized extreme values can be computed in amortized $O(1)$ time per target position using monotonic queues, preserving the amortized sublinear cost of batch lower-bound evaluation. We define the conservative upper and lower envelope bounds $\hat{\mathbb{U}}_i^{\max}$ and $\hat{\mathbb{L}}_i^{\min}$ for z-normalized matching:

$$\hat{\mathbb{U}}_i^{\max} = \max\left\{\frac{\mathbb{U}_i^T - \mu_i^{\min}}{\sigma_i^{\min}}, \frac{\mathbb{U}_i^T - \mu_i^{\min}}{\sigma_i^{\max}}\right\} \tag{33}$$

$$\hat{\mathbb{L}}_i^{\min} = \min\left\{\frac{\mathbb{L}_i^T - \mu_i^{\max}}{\sigma_i^{\min}}, \frac{\mathbb{L}_i^T - \mu_i^{\max}}{\sigma_i^{\max}}\right\} \tag{34}$$

For any non-degenerate candidate $X = T_{(k)}$ and relative position r, these bounds satisfy $\mathbb{U}_r^{\hat{X}} \leq \hat{\mathbb{U}}_{k+r-1}^{\max}$ and $\mathbb{L}_r^{\hat{X}} \geq \hat{\mathbb{L}}_{k+r-1}^{\min}$, preserving the lower-bound property. Using these envelope bounds, we define the normalized valley masks as:

$$\hat{g}_U(i,t) = \mathbf{1}_{\{t \geq \hat{\mathbb{U}}_i^{\max}\}}, \quad \hat{g}_L(i,t) = \mathbf{1}_{\{t \leq \hat{\mathbb{L}}_i^{\min}\}}. \tag{35}$$

With these definitions in place, the normalized masks $\hat{g}_U$ and $\hat{g}_L$ replace the raw masks g_U and g_L in the sliding inner-product formulation. This robust adaptation ensures that our amortized $O(\log m)$ lower-bound evaluation cost is preserved even when rigorous scale and baseline invariance are enforced. Unlike the target series, the query Q is z-normalized only once, keeping its elements q_i fixed throughout the search. Consequently, the penalty functions h_U and h_L are constructed exclusively from these static q_i values and the sweeping threshold t. All volatility introduced by the sliding window's shifting mean and variance is absorbed entirely by the target-side mask functions ($\hat{g}_U$ and $\hat{g}_L$). This isolation decouples sequence morphology from amplitude scaling.

5.2 The Cascaded FSMELD Pipeline

To optimize the tightness and pruning power of our lower bound (LB_{IP}), we leverage the heuristic threshold optimization technique detailed in Section 4.4 to compute proper "water level" thresholds (t_U, t_L). A single global threshold for a massive time series S often fails to capture local morphological variations, resulting in a loose lower bound; a strategic segmentation approach is essential. Rather than processing the entire series S at once, we divide it into overlapping macro-segments (denoted by superscripts, e.g., $T^{(i)}$). This segmentation strategy serves a dual purpose.

First, it allows for the calculation of locally optimized thresholds (t_U, t_L) based on the conservative envelope bounds, enabling the lower bound to tightly adapt to local baseline volatility. Second, because continuous lower-bound evaluation is mathematically

[4] Subsequences with zero or near-zero standard deviation are excluded from z-normalized search to avoid division by zero and numerical instability.

transformed into a sliding inner product using the aforementioned masks, it can be substantially accelerated via FFT. We set the segment length to $\ell = 2^{\lceil \log_2(8m) \rceil}$, using an empirical power-of-two FFT size while keeping $\ell = \Theta(m)$. Each segment therefore contains $\Theta(m)$ candidate windows over which the costs of threshold selection and sliding-inner-product evaluation are amortized.

Algorithm 3 solves the fixed-cutoff range-search problem in Definition 4 using a cascaded filter–verify pipeline. For each candidate, the FFT stage produces a raw penalty estimate $z_j = lb_j + lb'_j$, which is converted to the distance-space lower bound $b_j = \mathrm{trans}(z_j)$. Negative penalties are clamped to zero before square-root transformations.[5] A candidate with a finite bound is pruned when $b_j > \xi + \delta_{\mathrm{FFT}}$, where δ_{FFT} denotes the numerical allowance used in the FFT-stage comparison. The implementation uses a base tolerance parameter of 10^{-9}. Otherwise, the candidate proceeds to GLB filtering and, if necessary, exact elastic distance (ELD) verification. Candidate windows that cannot be safely z-normalized are excluded from the search. For simplicity, Algorithm 3 omits numerical safeguards, invalid-window checks, and standard tail handling.

Algorithm 3: FSMELD_normalized(S, Q, ξ)

Input: **Time series** $S \in \mathbb{R}^n$**, z-normalized query** $Q \in \mathbb{R}^m$ **and threshold** $\xi \geq 0$.
Output: $\{i \mid ELD(\hat{S}_{(i)}, Q) \leq \xi\}$.

$\mathbb{A} \leftarrow \emptyset$;
$\ell \leftarrow 2^{\lceil \log_2(8m) \rceil}$;
$n_{seg} \leftarrow \lceil \frac{n-m+1}{\ell-m+1} \rceil$; $N_w \leftarrow \ell - m + 1$;
foreach $i \in \{0, 1, \ldots, n_{seg} - 1\}$ **do**
  $T^{(i)} \leftarrow (S[i \times N_w + 1], S[i \times N_w + 2], \ldots, S[i \times N_w + \ell])$;
  $\hat{\mathbb{U}}^{\max}, \hat{\mathbb{L}}^{\min} \leftarrow$ computed from $T^{(i)}$ using Eqs. (33) and (34) ;
  $t_U \leftarrow \textsc{OptUpperThreshold}(\hat{\mathbb{U}}^{\max}, Q)$; // Alg. 2
  $t_L \leftarrow \textsc{OptLowerThreshold}(\hat{\mathbb{L}}^{\min}, Q)$;
  $A \leftarrow (\hat{g}_U(1, t_U), \hat{g}_U(2, t_U), \ldots, \hat{g}_U(\ell, t_U))$;
  $B \leftarrow (h_U(1, t_U), h_U(2, t_U), \ldots, h_U(m, t_U))$;
  $\{lb_j\}_{j=1}^{N_w} \leftarrow \mathrm{SIP}(A, B)$; // Alg. 1
  $A' \leftarrow (\hat{g}_L(1, t_L), \hat{g}_L(2, t_L), \ldots, \hat{g}_L(\ell, t_L))$;
  $B' \leftarrow (h_L(1, t_L), h_L(2, t_L), \ldots, h_L(m, t_L))$;
  $\{lb'_j\}_{j=1}^{N_w} \leftarrow \mathrm{SIP}(A', B')$;
  foreach $j \in \{1, 2, \ldots, N_w\}$ **do**
    if $\xi \geq \mathrm{trans}(lb_j + lb'_j) - \delta_{\mathrm{FFT}}$ **then**
      if $\xi \geq LB(\hat{S}_{(j+i\times N_w)}, Q)$ **then**
        if $\xi \geq ELD(\hat{S}_{(j+i\times N_w)}, Q)$ **then**
          $\mathbb{A} \leftarrow \mathbb{A} \cup \{j + i \times N_w\}$;
return $\mathbb{A}$;

6 EXPERIMENTS

Our experiments address four research questions (RQs):

- **RQ1.** How effective is the proposed lower bound (LB_{IP}) in terms of pruning rate and lower-bound evaluation cost? (Section 6.2)
- **RQ2.** How does FSMELD compare to state-of-the-art exact subsequence search pipelines in end-to-end efficiency? (Section 6.3)
- **RQ3.** How do the cascaded filters and heuristic threshold optimization each contribute to overall performance? (Section 6.4)
- **RQ4.** How robust is FSMELD to core parameter variations, such as the temporal constraint w and the search cutoff ξ? (Section 6.5)

6.1 Experimental Setup

Hardware and datasets. All algorithms are implemented in C++ and evaluated on a workstation equipped with a 2.00 GHz Intel Core Ultra 7 255H CPU, 32 GB of RAM, and a 1-TB SSD. We use six datasets, comprising five real-world time series and one synthetic random walk dataset. Table 5 summarizes their characteristics.

Table 5: Datasets used in the experiments.

Dataset	Length n	Description
ECG[6]	1.7×10^9	ECG signals from 15 subjects (1 kHz).
TEMP[7]	1.9×10^9	Monthly temperature records from China.
THCHS[8]	1.2×10^9	Mandarin speech from THCHS-30 (16 kHz).
PLA[9]	9.0×10^8	PLAID appliance current and voltage (30 kHz).
PLETH[10]	4.2×10^8	VTaC plethysmography (PPG, 250 Hz).
RW	1.0×10^9	Synthetic random walk.

Baselines. The main contribution of FSMELD is a unified framework for amortized sublinear lower-bound evaluation under ERP, EDR, LCSS, and MSM. Although the formulation also supports DTW, we do not claim a performance advantage over the DTW-specific method FSMDTW [6]. Our experiments focus on these four non-DTW measures to evaluate the practical benefits of the unified construction. UCR Suite supports ED/DTW subsequence search, MASS computes Euclidean distance profiles, and SWAMP addresses DTW motif discovery [1, 20, 32]. Their published formulations do not directly solve the four non-DTW range-search tasks evaluated here. We therefore construct comparable exact-search pipelines by applying GLB and applicable measure-specific lower bounds to the same sliding candidates, followed by exact verification. We compare LB_{IP} with GLB and the available measure-specific lower bounds for these four measures (Table 6). We measure their standalone lower-bound evaluation costs and compare the corresponding lower-bound-plus-ELD pipelines with FSMELD in exact search. All surviving candidates are verified using the same band-constrained dynamic programming.

Parameters and protocol. Unless stated otherwise, $m \in \{128, 256, 512, 1024, 2048, 4096\}$, $w = \lfloor 0.05m \rfloor$, and $\ell = 2^{\lceil \log_2(8m) \rceil}$. For each configuration, all methods share the same z-normalized query and candidate windows, w, and ξ. The cutoff ξ, computed offline as the tenth-smallest exact distance under the corresponding measure and constraint, remains fixed during search; all valid candidates with exact distance at most ξ are returned.

[5] For LCSS, the accumulated penalty z counts certified lost matches; hence $\mathrm{trans}(z) = z/m$, consistent with Eq. (6).

[6] https://physionet.org/content/butqdb/1.0.0/
[7] https://www.tpdc.ac.cn/zh-hans/data/71ab4677-b66c-4fd1-a004-b2a541c4d5bf
[8] https://www.openslr.org/18/
[9] https://figshare.com/articles/dataset/PLAID_-_A_Voltage_and_Current_Measurement_Dataset_for_Plug_Load_Appliance_Identification_in_Households/10084619
[10] https://physionet.org/content/vtac/1.0/

Table 6: Comparative lower bounds and default parameters.

Measure	Comparative Lower Bounds	Parameters
ERP	LB_{ERP} [7], $LB_{\text{Kim-ERP}}$ [7], $LB_{\text{Keogh-ERP}}$ [7], GLB_{ERP} [19]	$g = 0$
EDR	GLB_{EDR} [19]	$\epsilon = 0.1$
LCSS	LB_{LCSS} [25], GLB_{LCSS} [19]	$\epsilon = 0.2$
MSM	LB_{MSM} [24], GLB_{MSM} [19]	$c = 0.5$

6.2 Lower-Bound Effectiveness

We first examine whether LB_{IP} can discard a substantial fraction of candidates at low evaluation cost before evaluating the full pipeline.

Tightness and Pruning Power. We measure tightness as $TLB = LB_{IP}(X, Q)/ELD(X, Q)$, where $TLB \leq 1$ and larger values indicate tighter bounds. Table 7 reports the mean TLB and pruning rate (i.e., the fraction of candidates safely discarded) across the six datasets. A modest TLB can still permit pruning when $ELD(X, Q)$ is large relative to ξ, since only $LB_{IP}(X, Q) > \xi$ is required. LB_{IP} achieves an overall mean pruning rate of 64.5% across all elastic measures, increasing from 56.9% at $m = 128$ to 74.8% at $m = 4096$, demonstrating its effectiveness for long queries.

Table 7: Effectiveness of LB_{IP}. Results are arithmetic means over the six datasets. Overall additionally averages over the four elastic measures.

Statistic	Measure	Query Length m					
		128	**256**	**512**	**1024**	**2048**	**4096**
Mean TLB	ERP	0.243	0.272	0.223	0.224	0.246	0.258
	EDR	0.144	0.145	0.122	0.120	0.100	0.101
	LCSS	0.093	0.081	0.069	0.102	0.075	0.083
	MSM	0.048	0.050	0.047	0.058	0.048	0.047
	Overall	**0.132**	**0.137**	**0.115**	**0.126**	**0.117**	**0.122**
Mean Pruning Rate (%)	ERP	63.1	69.9	73.0	70.7	75.5	81.1
	EDR	57.8	64.3	69.0	72.9	75.4	83.0
	LCSS	73.0	68.4	71.4	61.8	64.3	71.3
	MSM	33.9	39.5	39.4	50.3	54.4	63.9
	Overall	**56.9**	**60.5**	**63.2**	**63.9**	**67.4**	**74.8**

Lower-Bound Evaluation Cost. To demonstrate the computational advantage of our amortized sublinear approach, we compare the online evaluation time of LB_{IP} against GLB and five metric-specific bounds. The timing includes all necessary steps for lower-bound computation per candidate (e.g., sliding statistics, normalization, envelope/mask construction, and SIP computation). As illustrated in Figure 4, the average cost of LB_{IP} remains low and stable, whereas the costs of competing $O(m)$ bounds increase linearly. Overall, LB_{IP} achieves average speedups of 46.1× over GLB and 12.0× over the metric-specific bounds.

6.3 End-to-End Search Efficiency

We next evaluate whether the theoretical speedups of LB_{IP} translate into practical end-to-end improvements in exact subsequence range search. The runtime encompasses data loading, online preprocessing, all filtering stages, and exact verification. As illustrated in Figure 5, FSMELD significantly outperforms GLB+ELD and other applicable metric-specific baselines. Specifically, FSMELD achieves an average speedup of 2.40× over GLB+ELD, with per-measure averages ranging from 1.68× to 2.81×. Furthermore, the performance gap widens as the query length m increases. This trend supports our theoretical claim that the amortized sublinear evaluation of LB_{IP} becomes increasingly beneficial for long sequence queries compared to traditional linear-time filters.

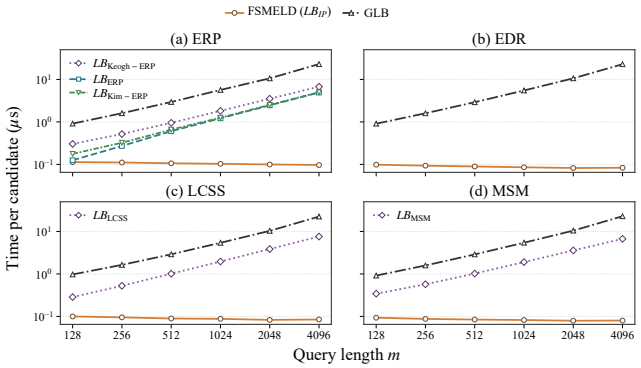


Figure 4: Mean online lower-bound evaluation time per candidate, averaged over the six datasets.

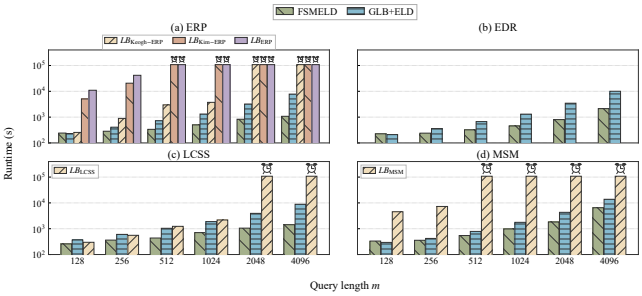


Figure 5: End-to-end runtime of FSMELD, GLB+ELD, and five metric-specific LB+ELD baselines for different query lengths.

6.4 Ablation Study

Contribution of the Cascaded Filters. We compare the complete cascade (LB_{IP}+GLB+ELD) with GLB+ELD and LB_{IP}+ELD. Table 8 shows that removing LB_{IP} increases mean runtime by a factor of 1.58–3.40, while removing GLB incurs a much larger increase. The latter result shows that cheap first-stage filtering alone is insufficient for this workload: the remaining candidates still require a stronger filter before exact verification. The cascade therefore combines inexpensive candidate rejection with selective verification.

Impact of Heuristic Threshold Optimization. Threshold selection must preserve useful pruning without offsetting the savings from batch evaluation. We compare the full objective $p_1(t)+p_2(t)+p_3(t)$ with the heuristic objective $p_1(t)$ from Section 4. In the example in Figure 6, the heuristic captures the full objective's high-value region and selects the same threshold. Table 9 evaluates this trade-off on ECG, reporting each measure and query length separately.

Table 8: Cascade ablation on ECG and RW. Runtime (s) is averaged over the two datasets and four measures.

	Query length m		
Variant	256	512	1024
Remove LB_{IP}	526.2	927.3	1673.8
Remove GLB	13976.6	43683.6	139555.3
FSMELD	**333.5**	**396.2**	**492.0**

Pruning rates differ by at most 0.2 percentage points, while the heuristic achieves a higher end-to-end speedup in every reported configuration. Omitting the boundary terms $p_2(t)$ and $p_3(t)$ thus improves the observed cost–pruning trade-off without changing the exactness of the range search.

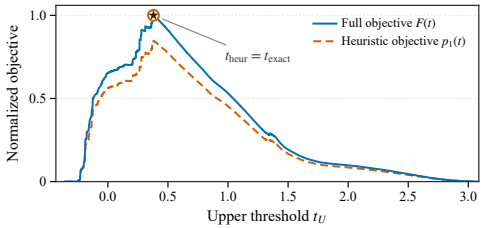


Figure 6: Full and heuristic upper-threshold objectives for an ECG macro-segment. Both objectives are normalized by $F(t_{\text{exact}})$.

Table 9: Comparison of exact and heuristic threshold optimization on ECG. Pruning rates are for LB_{IP}; end-to-end speedups are relative to GLB+ELD.

Measure	m	Pruning rate (%)		Speedup (×)	
		Exact	**Heuristic**	**Exact**	**Heuristic**
ERP	256	74.32	74.33	0.57	**1.60**
	512	75.33	75.39	0.73	**2.38**
	1024	89.27	89.33	0.94	**5.00**
	2048	90.21	90.27	1.14	**7.21**
EDR	256	86.09	86.13	0.64	**1.98**
	512	92.16	92.17	0.82	**3.81**
	1024	94.40	94.43	1.11	**6.85**
	2048	95.06	95.08	1.34	**10.81**
LCSS	256	87.33	87.32	0.89	**2.46**
	512	93.11	93.19	1.19	**4.74**
	1024	94.30	94.34	1.53	**7.27**
	2048	94.91	94.94	1.92	**11.05**
MSM	256	59.19	59.07	0.59	**1.48**
	512	68.69	68.72	0.68	**2.15**
	1024	90.22	90.23	1.00	**5.72**
	2048	90.36	90.36	1.28	**7.91**

6.5 Parameter Sensitivity

Sensitivity to Temporal Constraint w. To evaluate the impact of the temporal constraint w, we fix $m = 1024$ and vary the ratio $w/m \in \{0.01, 0.02, 0.03, 0.05, 0.10\}$ on the ECG and RW datasets across all four elastic measures. For each configuration, the cutoff is recalibrated to the tenth-smallest exact distance. As shown in Figure 7(a), FSMELD achieves an average speedup greater than one over the GLB+ELD pipeline for each measure across the entire tested range. While the relative speedup generally decreases as a larger w relaxes the warping constraint, FSMELD maintains a clear efficiency advantage over GLB+ELD.

Sensitivity to Search Cutoff ξ. We use fixed excerpts containing $K = 10^7$ candidate windows per dataset. The cutoff ξ is set to the $\lceil qK \rceil$-th smallest exact distance for $q \in \{10^{-5}\%, 10^{-4}\%, 5 \times 10^{-4}\%, 10^{-3}\%, 10^{-2}\%\}$. Figure 7(b) shows the largest speedups at the most selective cutoff, ranging from 3.41× to 5.11× across the four measures. At looser cutoffs, the speedups approach one. These results indicate that the first-stage filter provides its greatest benefit when the cutoff is small and the range query is highly selective.

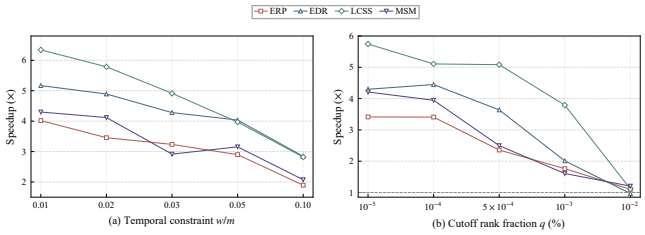


Figure 7: Parameter sensitivity of FSMELD: (a) temporal constraint w/m; (b) cutoff rank fraction q. Each point shows the average end-to-end speedup over GLB+ELD.

7 CONCLUSION

In this paper, we introduced FSMELD, an exact filter–verify framework for subsequence matching under DTW, ERP, EDR, LCSS, and MSM. Its LB_{IP} formulation decomposes envelope penalties into sliding inner products, enabling FFT-based evaluation in amortized $O(\log m)$ time per candidate. Experiments on six datasets demonstrate that FSMELD achieves high pruning rates and outperforms state-of-the-art exact baselines, providing a practical framework for large-scale time series analytics.

A CORRECTNESS OF THE LCSS INSTANTIATION

For equal-length sequences $X, Q \in \mathbb{R}^m$, let $\epsilon \geq 0$ be the LCSS matching tolerance and let $t_U, t_L \in \mathbb{R}$ be arbitrary thresholds. We use the masks and binary penalties of the LCSS instantiation in the main paper.

Consider the upper-envelope case. If $\mathbb{U}_i^X \leq t_U$ and $q_i > t_U + \epsilon$, then

$$q_i > t_U + \epsilon \geq \mathbb{U}_i^X + \epsilon. \tag{36}$$

According to the temporal constraint of LCSS, all temporally admissible target elements associated with the i-th query position are bounded above by $\mathbb{U}_i^X$. Therefore, $g_U(i, t_U)h_U(i, t_U) = 1$ certifies that the corresponding query position cannot participate in an admissible LCSS match. The lower-envelope case follows symmetrically.

Let the total number of certified lost matches be

$$K_{\mathrm{LCSS}} = \sum_{i=1}^{m} g_U(i, t_U) h_U(i, t_U) + \sum_{i=1}^{m} g_L(i, t_L) h_L(i, t_L). \tag{37}$$

Because $\mathbb{U}_i^X \geq \mathbb{L}_i^X$ and $\epsilon \geq 0$, the upper- and lower-envelope conditions cannot simultaneously certify the same query position as a lost match: the former requires $q_i > \mathbb{U}_i^X + \epsilon$, whereas the latter requires $q_i < \mathbb{L}_i^X - \epsilon$. Hence, K_{LCSS} counts distinct query positions that cannot be matched. Since an LCSS matching uses each query position at most once, we obtain

$$\mathrm{LCSS}(X, Q) \leq m - K_{\mathrm{LCSS}}. \tag{38}$$

Using the normalized LCSS distance, we obtain

$$d_{\mathrm{LCSS}}(X, Q) = 1 - \frac{\mathrm{LCSS}(X, Q)}{m} \geq \frac{K_{\mathrm{LCSS}}}{m} = LB_{IP}(X, Q, t_U, t_L), \tag{39}$$

where the last equality uses $\mathrm{trans}(z) = z/m$.

The same reasoning applies to any conservative envelopes that contain all temporally admissible target values, including the shared envelopes used for z-normalized candidates.

B OPTIMIZATION OF LB_{IP} FOR A SINGLE SUBSEQUENCE PAIR

Theorem 4. *Given a query sequence $Q \in \mathbb{R}^m$ and a specific candidate subsequence $X \in \mathbb{R}^m$, the exact thresholds (t_U^*, t_L^*) that maximize the proposed inner-product lower bound $LB_{IP}(X, Q, t_U, t_L)$ can be computed in $O(m \log m)$ time.*

Proof. Without loss of generality, we focus on computing the optimal upper threshold t_U^*, since the derivation for t_L^* follows symmetrically.

By the definition of the LB_{IP} framework, the threshold t_U independently dictates the penalty term associated with the upper envelope. Let $F_U(t_U)$ denote the threshold-dependent objective function to be maximized within LB_{IP}. For the established elastic distances, $F_U(t_U)$ can be formulated as an element-wise summation over the sequence length m:

$$F_U(t_U) = \sum_{i=1}^{m} \phi_i(t_U) \tag{40}$$

where $\phi_i(t_U) = g_U(i, t_U) h_U^{\mu}(i, t_U)$. Depending on the elastic measure μ, ϕ_i may be piecewise constant, piecewise linear, or piecewise quadratic. Let $\mathbb{B}_U$ contain all envelope-activation values $\mathbb{U}_i^X$ and all metric-specific query-state transition values, such as q_i and $\tau_\mu(q_i)$. Since each element contributes only a constant number of events, $|\mathbb{B}_U| = O(m)$.

By Theorem 2, while the target-side mask remains fixed, every active upper-side penalty is non-increasing as t_U increases. Therefore, a maximizer can be selected from the envelope-activation set $\{\mathbb{U}_i^X\}_{i=1}^m$. The additional query-state events in $\mathbb{B}_U$ are used only to update the response-function statistics during the sweep.

A naive evaluation of $F_U(b)$ for each candidate $b \in \mathbb{B}_U$ requires $O(m^2)$ time. However, this process can be optimized to $O(m \log m)$ using a two-phase sweep-line algorithm:

(1) **Sorting Phase:** We extract all $O(m)$ breakpoints into the set $\mathbb{B}_U$ and sort them in ascending order. This step requires $O(m \log m)$ time.
(2) **Sweep-line Evaluation Phase:** We maintain metric-specific sufficient statistics while sweeping through the sorted events. DTW and ERP maintain zeroth-, first-, and second-order moments; EDR and LCSS maintain counts; and MSM maintains counts and first-order moments for its linear and saturated states. Each query element changes state only a constant number of times, and each transition updates a constant number of statistics. Consequently, after sorting, all candidate envelope thresholds can be evaluated in $O(m)$ time.

Since the sweep-line evaluation processes the $O(m)$ breakpoints in $O(m)$ time, the overall computational complexity is dominated by the sorting phase, resulting in $O(m \log m)$. The identical algorithmic logic applies symmetrically to the derivation of t_L^*, thereby concluding the proof. □

C EXACT THRESHOLD OPTIMIZATION FOR A TARGET SEGMENT

This appendix gives the complete algorithm underlying Theorem 3. We retain the notation and query-state representation of Section 4.4. The exact procedure maximizes $p_1(t) + p_2(t) + p_3(t)$, using the same two heaps and query moments as the heuristic, together with additional statistics for the boundary terms.

C.1 Objective and Boundary Statistics

Let $\mathcal{U} \in \mathbb{R}^\ell$ be the shared upper envelope and $Q \in \mathbb{R}^m$ the query, with $\ell \geq 2m$. For the raw formulation, $\mathcal{U} = \mathbb{U}^T$, and $g_U(j, t) = \mathbf{1}_{\{\mathcal{U}_j \leq t\}}$. The functions h_U, D, and τ, and the moment order p, are those used in Section 4.4.2. In particular, the LCSS convention $D(q_i) = 1$ denotes a certified lost match in the threshold-selection procedure.

Recall that H_2 stores query indices with $t < \tau(q_i)$, whereas H_1 stores those with $\tau(q_i) \leq t < q_i$. Indices with $q_i \leq t$ belong to neither heap. The unweighted statistics remain

$$\mathcal{M}_\alpha = \sum_{i \in H_1} q_i^\alpha, \qquad \sigma_D = \sum_{i \in H_2} D(q_i), \tag{41}$$

where $\mathcal{M}_0 = |H_1|$ and $\alpha = 0, \ldots, p$. They evaluate the total query penalty as

$$\Delta_Q(Q, t) = \sigma_D + f_p(\mathcal{M}_p, \ldots, \mathcal{M}_1, |H_1|, t). \tag{42}$$

As in the heuristic, G_U counts active core positions $m \leq j \leq \ell - m + 1$, giving $p_1(t) = G_U \Delta_Q(Q, t)$.

For query position i, a target position j participates in the cumulative objective precisely when $i \leq j \leq \ell - m + i$. Its associated boundary positions are therefore

$$\mathcal{B}_i = \{i, \ldots, m - 1\} \cup \{\ell - m + 2, \ldots, \ell - m + i\}. \tag{43}$$

An integer range whose lower endpoint exceeds its upper endpoint is empty. Thus $|\mathcal{B}_i| = m - 1$, including the empty-boundary case

$m = 1$. These sets are represented by their two index ranges; their elements need not be stored separately.

Reordering the boundary contributions in Eq. (26) gives

$$p_2(t) + p_3(t) = \sum_{i=1}^{m} h_U(i,t) \sum_{j \in \mathcal{B}_i} g_U(j,t). \tag{44}$$

We maintain the corresponding boundary-weighted statistics:

$$\begin{aligned} \mathcal{M}_\alpha^B &= \sum_{i \in H_1} q_i^\alpha \sum_{j \in \mathcal{B}_i} g_U(j,t), \\ \sigma_D^B &= \sum_{i \in H_2} D(q_i) \sum_{j \in \mathcal{B}_i} g_U(j,t). \end{aligned} \tag{45}$$

The superscript B identifies boundary statistics. In particular, $\mathcal{M}_0^B$ counts active boundary contributions associated with indices in H_1; it is not the cardinality of H_1.

The same polynomial f_p evaluates the boundary contribution:

$$\begin{aligned} p_2(t) + p_3(t) = \sigma_D^B \\ + f_p(\mathcal{M}_p^B, \ldots, \mathcal{M}_1^B, \mathcal{M}_0^B, t). \end{aligned} \tag{46}$$

This follows because f_p is linear in its moment and count arguments. For example, its boundary-weighted value for ERP and DTW is $\mathcal{M}_2^B - 2t\mathcal{M}_1^B + t^2\mathcal{M}_0^B$. For EDR and LCSS, the contribution of H_1 remains zero: $f_0(\mathcal{M}_0^B, t) = 0$. Positive-order moment arguments are omitted when $p = 0$.

C.2 Exact Threshold-Selection Procedure

Algorithm 4 constructs $\mathcal{T}$ from all distinct values of $\mathcal{U}$ and records the target positions associated with each value. It initializes the query states at $t_0 = \mathcal{T}_1$ as in Algorithm 2. Initially, G_U and all boundary statistics are zero because no target masks have yet been activated.

At each candidate threshold t, the procedure first processes all query-state transitions up to t. At this stage, the active target positions are exactly those with $\mathcal{U}_j < t$. For a transitioning query index k, let

$$b = \sum_{j \in \mathcal{B}_k} \mathbf{1}_{\{\mathcal{U}_j < t\}}. \tag{47}$$

The value b is computed by scanning the two ranges in Eq. (43), in $O(m)$ time. When k moves from H_2 to H_1, its fixed contributions $D(q_k)$ and $bD(q_k)$ are removed from σ_D and σ_D^B, respectively. Its moments are then added to $\mathcal{M}_\alpha$ and, with multiplicity b, to $\mathcal{M}_\alpha^B$. When k leaves H_1, these moments are removed.

The procedure next activates all target positions with $\mathcal{U}_j = t$. A core position increments G_U. For a boundary position j, the associated query indices are

$$\max\{1, j - \ell + m\} \le k \le \min\{m, j\}. \tag{48}$$

For each such k, a fixed-penalty state contributes $D(q_k)$ to σ_D^B, a post-transition state contributes q_k^α to each $\mathcal{M}_\alpha^B$, and a submerged state contributes zero.

The strict inequality in Eq. (47) is essential: masks with $\mathcal{U}_j = t$ are added only in the subsequent activation phase. Query transitions use $\tau(q_k) \le t$ and $q_k \le t$, so the zero penalty at $t = q_k - \epsilon$ is handled correctly for EDR and LCSS. Query-only breakpoints need not be evaluated as candidates; the heaps process all transitions crossed before the next envelope threshold.

Here, LB_i is the full cumulative raw-penalty objective at $t = \mathcal{T}_i$. The groups of target positions are stored alongside their sorted envelope values; retrieving a group does not require scanning $\mathcal{U}$ again.

C.3 Correctness and Complexity

Theorem 5. *Algorithm 4 returns a threshold maximizing $p_1(t) + p_2(t) + p_3(t)$ for the supplied shared envelope and query. Its running time is $O(m^2 + \ell \log \ell)$ and its auxiliary space is $O(\ell + m)$.*

Proof. We show that after processing a candidate threshold t, the heaps represent the query states at t, G_U counts the active core masks, and the boundary statistics satisfy Eq. (45).

At t_0, the query states and unweighted moments are initialized directly. There are no envelope values below t_0, so all boundary statistics are initially zero. Activating the positions with envelope value t_0 establishes the stated invariant.

For a subsequent candidate t, all previously active target positions have $\mathcal{U}_j < t$, and no other target positions have yet been activated. Thus the value b in Eq. (47) gives exactly the multiplicity with which the transitioning query element contributes to the boundary terms. The two while loops update its unweighted and boundary-weighted contributions together. Since $\tau(q_k) \le q_k$, transferring elements from H_2 before removing elements from H_1 also handles a query that crosses both transitions between successive envelope thresholds.

The activation phase then adds each newly active core or boundary position exactly once, using the query states already updated to t. The invariant therefore holds after this phase. Equations (44) and (46) imply that LB_i equals $p_1(t) + p_2(t) + p_3(t)$. By Theorem 2, evaluating all distinct envelope values includes an exact maximizer.

Sorting the envelope values and grouping their positions costs $O(\ell \log \ell)$ time. Each query element enters and leaves each heap at most once, giving $O(m \log m)$ heap maintenance. There are at most $2m$ query-state transitions that require a boundary scan, and each scan examines $m - 1$ positions, for $O(m^2)$ total work. During boundary activation, the total number of associated query positions visited is

$$\sum_{j=1}^{m-1} j + \sum_{j=\ell-m+2}^{\ell} (\ell - j + 1) = m(m-1).$$

Each visit updates at most $p+1$ moments, where $p \le 2$ is a measure-specific constant. Core activations and final objective evaluations cost $O(\ell)$ in total. Hence the overall running time is $O(m^2 + \ell \log \ell)$.

The sorted position groups and recorded objective values use $O(\ell)$ space, and the heaps use $O(m)$ space. The sets $\mathcal{B}_i$ are enumerated from their two index ranges without storing all boundary pairs. The auxiliary space is therefore $O(\ell + m)$. □

C.4 Computing the Lower Threshold

The lower threshold can be obtained by reflecting the input values and reusing the same procedure. Given the shared lower envelope $\mathcal{L}$, compute

$$t_L^* = -\textsc{ExactUpperThreshold}(-\mathcal{L}, -Q). \tag{49}$$

For ERP, the reflected call also replaces the gap value g by $-g$; the parameters ϵ and c remain unchanged for EDR, LCSS, and MSM. Under this reflection, $\mathcal{L}_j \geq t$ is equivalent to $-\mathcal{L}_j \leq -t$, and each lower-side penalty equals the corresponding upper-side penalty for the reflected query. The returned threshold therefore maximizes the cumulative lower penalty. Computing both thresholds retains the $O(m^2 + \ell \log \ell)$ time bound.

D FSMELD FOR RAW SUBSEQUENCE MATCHING

While the main text of this paper focuses on subsequence matching under z-score normalization—the standard prerequisite for handling baseline and scale variations in time series queries—the FSMELD framework is natively applicable to raw subsequence matching. In this appendix, we detail the algorithm when executed directly on unnormalized time series data.

Compared to the normalized version, raw subsequence matching completely bypasses the dynamic bounding estimations for local means and standard deviations. Consequently, the segment upper and lower envelopes ($\mathbb{U}^{(i)}$ and $\mathbb{L}^{(i)}$) can be utilized directly as static boundaries, making the LB_{IP} formulation even more straightforward. The complete procedure is presented in Algorithm 5.

Strategic Macro-Segmentation and Envelopes. Because a single global threshold for a massive time series $S \in \mathbb{R}^n$ fails to capture local morphological variations, FSMELD divides the long target series S into overlapping macro-segments. As shown in Algorithm 5 (Lines 2-3), the segment length is defined as $\ell = 2^{\lceil \log_2(8m) \rceil}$, which utilizes a power-of-two size to optimize the internal Fast Fourier Transform (FFT) operations while strictly keeping $\ell = \Theta(m)$. The total number of segments is $n_{seg} = \lceil \frac{n-m+1}{\ell-m+1} \rceil$, with adjacent segments overlapping by $m-1$ points to ensure no candidate windows are missed at the boundaries. For each segment $T^{(i)}$, we directly compute its raw upper and lower envelopes $\mathbb{U}^{(i)}$ and $\mathbb{L}^{(i)}$ based on the temporal constraint w.

Threshold Optimization and Vector Construction. Because raw sequence matching does not involve shifting means and variances, the heuristic threshold optimization procedures (OptUpperThreshold and OptLowerThreshold) are directly applied to the raw envelopes (Lines 7-8). Once the locally optimal thresholds t_U and t_L are selected for the current segment, the target series is transformed into binary valley mask arrays (A and A'), and the query sequence Q is transformed into numeric protrusion penalty arrays (B and B').

Amortized Lower Bound Evaluation via FFT.. With the morphological alignment decoupled into the aforementioned vector representations, evaluating the lower bound across all $N_w = \ell - m + 1$ overlapping candidate subsequences within the segment mathematically equates to calculating the discrete linear convolutions. We encapsulate this operation into the Fast Sliding Inner Product (SIP) function (Lines 11 and 14). By leveraging the Convolution Theorem, the SIP function computes all inner products for the segment in $O(\ell \log \ell)$ time. Since each segment evaluates $\Theta(m)$ candidates simultaneously, the amortized cost of this lower bound evaluation is rigorously reduced to $O(\log m)$ per candidate window.

Cascaded Verification Pipeline. To guarantee no false dismissals while strictly minimizing the number of exact dynamic-programming computations, FSMELD relies on a cascaded filtering architecture (Lines 15-21). For any candidate subsequence $S_{(j+i\times N_w)}$, its inner-product lower bound is first checked against the cutoff ξ. To ensure numerical safety, this pruning comparison is offset by a conservative tolerance δ_{FFT}, which accounts for floating-point roundoff errors introduced during the FFT convolutions. Candidates surviving this initial LB_{IP} filter are then verified by a tighter, linear-time bound (i.e., *GLB*). Only candidates that pass both filtering stages undergo exact elastic distance evaluation (*ELD*).

Algorithm 4: ExactUpperThreshold($\mathcal{U}, Q$)

Input: $\mathcal{U} \in \mathbb{R}^{\ell}$, $Q \in \mathbb{R}^{m}$, $(\ell \geq 2m)$.
Output: An exact optimal upper threshold t_U^*.

$\mathcal{T} \leftarrow$ Extract unique elements from $\mathcal{U}$ and record their position groups ;
Sort $\mathcal{T}$ in ascending order ;
$H_1, H_2 \leftarrow$ Initialize two empty min-heaps ;
$G_U \leftarrow 0$; $t_0 \leftarrow \mathcal{T}_1$; $\sigma_D, \sigma_D^B \leftarrow 0$;
$\mathcal{M}_\alpha, \mathcal{M}_\alpha^B \leftarrow 0$ for $\alpha = 0, \ldots, p$;
for $j = 1$ *to* m **do**
  if $q_j \leq t_0$ **then**
    continue ; // $j \notin H_1 \cup H_2$
  if $\tau(q_j) > t_0$ **then**
    $H_2.insert(\tau(q_j), j)$; // Tuple of (thresh, idx)
    $\sigma_D \leftarrow \sigma_D + D(q_j)$;
  else
    $H_1.insert(q_j, j)$; // Tuple of (thresh, idx)
    for $\alpha = 0$ *to* p **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha + (q_j)^\alpha$;
for $i = 1$ *to* $|\mathcal{T}|$ **do**
  $t \leftarrow \mathcal{T}_i$;
  while ***not*** $H_2.empty()$ ***and*** $H_2.top().thresh \leq t$ **do**
    $k \leftarrow H_2.top()$.idx ;
    $b \leftarrow \sum_{j \in \mathcal{B}_k} \mathbf{1}_{\{\mathcal{U}_j < t\}}$;
    $\sigma_D \leftarrow \sigma_D - D(q_k)$; $\sigma_D^B \leftarrow \sigma_D^B - bD(q_k)$;
    $H_1.insert(q_k, k)$; // $H_1 \leftarrow H_1 \cup \{k\}$
    for $\alpha = 0$ *to* p **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha + (q_k)^\alpha$; $\mathcal{M}_\alpha^B \leftarrow \mathcal{M}_\alpha^B + b(q_k)^\alpha$;
    $H_2.pop()$;
  while ***not*** $H_1.empty()$ ***and*** $H_1.top().thresh \leq t$ **do**
    $k \leftarrow H_1.top()$.idx ;
    $b \leftarrow \sum_{j \in \mathcal{B}_k} \mathbf{1}_{\{\mathcal{U}_j < t\}}$;
    for $\alpha = 0$ *to* p **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha - (q_k)^\alpha$; $\mathcal{M}_\alpha^B \leftarrow \mathcal{M}_\alpha^B - b(q_k)^\alpha$;
    $H_1.pop()$;
  foreach j *in the recorded group with* $\mathcal{U}_j = t$ **do**
    if $m \leq j \leq \ell - m + 1$ **then**
      $G_U \leftarrow G_U + 1$;
    else
      for $k = \max\{1, j - \ell + m\}$ *to* $\min\{m, j\}$ **do**
        if $q_k \leq t$ **then**
          continue ; // Zero contribution.
        if $\tau(q_k) > t$ **then**
          $\sigma_D^B \leftarrow \sigma_D^B + D(q_k)$;
        else
          for $\alpha = 0$ *to* p **do**
            $\mathcal{M}_\alpha^B \leftarrow \mathcal{M}_\alpha^B + (q_k)^\alpha$;
  $\Delta_Q \leftarrow \sigma_D + f_p(\mathcal{M}_p, \ldots, \mathcal{M}_1, |H_1|, t)$;
  $LB_i \leftarrow G_U \times \Delta_Q + \sigma_D^B + f_p(\mathcal{M}_p^B, \ldots, \mathcal{M}_1^B, \mathcal{M}_0^B, t)$;
$idx \leftarrow \arg\max_{1 \leq i \leq |\mathcal{T}|} LB_i$;
return $\mathcal{T}_{idx}$; // Choose the first maximizer in case of ties.

Algorithm 5: FSMELD_raw(S, Q, ξ)

Input: Time series $S \in \mathbb{R}^n$, query $Q \in \mathbb{R}^m$ and threshold $\xi \geq 0$.
Output: $\{i \mid ELD(S_{(i)}, Q) \leq \xi\}$.

$\mathbb{A} \leftarrow \emptyset$;
$\ell \leftarrow 2^{\lceil \log_2(8m) \rceil}$;
$n_{seg} \leftarrow \lceil \frac{n-m+1}{\ell-m+1} \rceil$; $N_w \leftarrow \ell - m + 1$;
foreach $i \in \{0, 1, \ldots, n_{seg} - 1\}$ **do**
  $T^{(i)} \leftarrow (S[i \times N_w + 1], S[i \times N_w + 2], \ldots, S[i \times N_w + \ell])$;
  $\mathbb{U}^{(i)}, \mathbb{L}^{(i)} \leftarrow$ the upper and lower envelopes for $T^{(i)}$;
  $t_U \leftarrow$ OptUpperThreshold($\mathbb{U}^{(i)}, Q$) ; // Alg. 2
  $t_L \leftarrow$ OptLowerThreshold($\mathbb{L}^{(i)}, Q$) ;
  $A \leftarrow (g_U(1, t_U), g_U(2, t_U), \ldots, g_U(\ell, t_U))$;
  $B \leftarrow (h_U(1, t_U), h_U(2, t_U), \ldots, h_U(m, t_U))$;
  $\{lb_j\}_{j=1}^{N_w} \leftarrow \mathrm{SIP}(A, B)$; // Alg. 1
  $A' \leftarrow (g_L(1, t_L), g_L(2, t_L), \ldots, g_L(\ell, t_L))$;
  $B' \leftarrow (h_L(1, t_L), h_L(2, t_L), \ldots, h_L(m, t_L))$;
  $\{lb'_j\}_{j=1}^{N_w} \leftarrow \mathrm{SIP}(A', B')$;
  foreach $j \in \{1, 2, ..., N_w\}$ **do**
    if $\xi \geq \mathrm{trans}(lb_j + lb'_j) - \delta_{\mathrm{FFT}}$ **then**
      if $\xi \geq LB(S_{(j+i\times N_w)}, Q)$ **then**
        if $\xi \geq ELD(S_{(j+i\times N_w)}, Q)$ **then**
          $\mathbb{A} \leftarrow \mathbb{A} \cup \{j + i \times N_w\}$;
return $\mathbb{A}$;

REFERENCES

[1] Sara Alaee, Kaveh Kamgar, and Eamonn Keogh. 2020. Matrix Profile XXII: Exact Discovery of Time Series Motifs Under DTW. In *2020 IEEE International Conference on Data Mining (ICDM)*. IEEE, 900–905. https://doi.org/10.1109/ICDM50108.2020.00099

[2] Donald J. Berndt and James Clifford. 1994. Using Dynamic Time Warping to Find Patterns in Time Series. In *Knowledge Discovery in Databases: Papers from the 1994 AAAI Workshop (AAAI Technical Report)*, Usama M. Fayyad and Ramasamy Uthurusamy (Eds.). AAAI Press, Menlo Park, CA, USA, 359–370. https://aaai.org/papers/359-ws94-03-031/

[3] Karl Bringmann and Marvin Künnemann. 2015. Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping. In *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE, Piscataway, NJ, USA, 79–97. https://doi.org/10.1109/FOCS.2015.15

[4] Zemin Chao, Hong Gao, Yinan An, and Jianzhong Li. 2022. The Inherent Time Complexity and an Efficient Algorithm for Subsequence Matching Problem. *Proceedings of the VLDB Endowment* 15, 7 (2022), 1453–1465. https://doi.org/10.14778/3523210.3523222

[5] Zemin Chao, Hong Gao, Dongjing Miao, Jianzhong Li, and Hongzhi Wang. 2025. An Amortized $O(1)$ Lower Bound for Dynamic Time Warping in Motif Discovery. *IEEE Transactions on Knowledge and Data Engineering* 37, 5 (2025), 2239–2252. https://doi.org/10.1109/TKDE.2025.3544751

[6] Zemin Chao, Qiaoyi Zheng, Zhixin Qi, and Hongzhi Wang. 2025. FSMDTW: A Fast Index-Free Subsequence Matching Algorithm for Dynamic Time Warping. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3628–3640. https://doi.org/10.14778/3748191.3748220

[7] Lei Chen and Raymond Ng. 2004. On the Marriage of L_p-Norms and Edit Distance. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases*. Morgan Kaufmann, San Francisco, CA, USA, 792–803. https://doi.org/10.1016/B978-012088469-8.50070-X

[8] Lei Chen, M. Tamer Özsu, and Vincent Oria. 2005. Robust and Fast Similarity Search for Moving Object Trajectories. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 491–502. https://doi.org/10.1145/1066157.1066213

[9] Jens E. d'Hondt, Haojun Li, Fan Yang, Odysseas Papapetrou, and John Paparrizos. 2025. A Structured Study of Multivariate Time-Series Distance Measures. *Proceedings of the ACM on Management of Data* 3, 3, Article 121 (2025), 29 pages. https://doi.org/10.1145/3725258

[10] Hui Ding, Goce Trajcevski, Peter Scheuermann, Xiaoyue Wang, and Eamonn J. Keogh. 2008. Querying and Mining of Time Series Data: Experimental Comparison of Representations and Distance Measures. *Proceedings of the VLDB Endowment* 1, 2 (2008), 1542–1552. https://doi.org/10.14778/1454159.1454226

[11] Christos Faloutsos, M. Ranganathan, and Yannis Manolopoulos. 1994. Fast Subsequence Matching in Time-Series Databases. *ACM SIGMOD Record* 23, 2 (1994), 419–429. https://doi.org/10.1145/191843.191925

[12] Matthieu Herrmann and Geoffrey I. Webb. 2021. Early Abandoning and Pruning for Elastic Distances Including Dynamic Time Warping. *Data Mining and Knowledge Discovery* 35, 6 (2021), 2577–2601. https://doi.org/10.1007/s10618-021-00782-4

[13] Christopher Holder, Matthew Middlehurst, and Anthony Bagnall. 2024. A Review and Evaluation of Elastic Distance Functions for Time Series Clustering. *Knowledge and Information Systems* 66, 2 (2024), 765–809. https://doi.org/10.1007/s10115-023-01952-0

[14] Eamonn Keogh and Shruti Kasetty. 2002. On the Need for Time Series Data Mining Benchmarks: A Survey and Empirical Demonstration. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 102–111. https://doi.org/10.1145/775047.775062

[15] Eamonn Keogh and Chotirat Ann Ratanamahatana. 2005. Exact Indexing of Dynamic Time Warping. *Knowledge and Information Systems* 7, 3 (2005), 358–386. https://doi.org/10.1007/s10115-004-0154-9

[16] Sang-Wook Kim, Sanghyun Park, and Wesley W. Chu. 2001. An Index-Based Approach for Similarity Search Supporting Time Warping in Large Sequence Databases. In *Proceedings of the 17th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 607–614. https://doi.org/10.1109/ICDE.2001.914875

[17] Daniel Lemire. 2009. Faster Retrieval with a Two-Pass Dynamic-Time-Warping Lower Bound. *Pattern Recognition* 42, 9 (2009), 2169–2180. https://doi.org/10.1016/j.patcog.2008.11.030

[18] Jason Lines and Anthony Bagnall. 2015. Time Series Classification with Ensembles of Elastic Distance Measures. *Data Mining and Knowledge Discovery* 29, 3 (2015), 565–592. https://doi.org/10.1007/s10618-014-0361-2

[19] John Paparrizos, Kaize Wu, Aaron J. Elmore, Christos Faloutsos, and Michael J. Franklin. 2023. Accelerating Similarity Search for Elastic Measures: A Study and New Generalization of Lower Bounding Distances. *Proceedings of the VLDB Endowment* 16, 8 (2023), 2019–2032. https://doi.org/10.14778/3594512.3594530

[20] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 262–270. https://doi.org/10.1145/2339530.2339576

[21] Ahmed Shifaz, Charlotte Pelletier, François Petitjean, and Geoffrey I. Webb. 2023. Elastic Similarity and Distance Measures for Multivariate Time Series. *Knowledge and Information Systems* 65, 6 (2023), 2665–2698. https://doi.org/10.1007/s10115-023-01835-4

[22] Alexandra Stefan, Vassilis Athitsos, and Gautam Das. 2012. The Move-Split-Merge Metric for Time Series. *IEEE Transactions on Knowledge and Data Engineering* 25, 6 (2012), 1425–1438. https://doi.org/10.1109/TKDE.2012.88

[23] Chang Wei Tan, François Petitjean, and Geoffrey I. Webb. 2019. Elastic Bands across the Path: A New Framework and Method to Lower Bound DTW. In *Proceedings of the 2019 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 522–530. https://doi.org/10.1137/1.9781611975673.59

[24] Chang Wei Tan, François Petitjean, and Geoffrey I. Webb. 2020. FastEE: Fast Ensembles of Elastic Distances for Time Series Classification. *Data Mining and Knowledge Discovery* 34, 1 (2020), 231–272. https://doi.org/10.1007/s10618-019-00663-x

[25] Michail Vlachos, Marios Hadjieleftheriou, Dimitrios Gunopulos, and Eamonn Keogh. 2003. Indexing Multi-Dimensional Time-Series with Support for Multiple Distance Measures. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 216–225. https://doi.org/10.1145/956750.956777

[26] Michail Vlachos, George Kollios, and Dimitrios Gunopulos. 2002. Discovering Similar Multidimensional Trajectories. In *Proceedings of the 18th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 673–684. https://doi.org/10.1109/ICDE.2002.994784

[27] Xiaoyue Wang, Abdullah Mueen, Hui Ding, Goce Trajcevski, Peter Scheuermann, and Eamonn J. Keogh. 2013. Experimental Comparison of Representation Methods and Distance Measures for Time Series Data. *Data Mining and Knowledge Discovery* 26, 2 (2013), 275–309. https://doi.org/10.1007/s10618-012-0250-5

[28] Geoffrey I. Webb and François Petitjean. 2021. Tight Lower Bounds for Dynamic Time Warping. *Pattern Recognition* 115 (2021), 107895. https://doi.org/10.1016/j.patcog.2021.107895

[29] Di Yao, Gao Cong, Chao Zhang, Xuying Meng, Rongchang Duan, and Jingping Bi. 2022. A Linear Time Approach to Computing Time Series Similarity Based on Deep Metric Learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2022), 4554–4571. https://doi.org/10.1109/TKDE.2020.3047070

[30] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. 2016. Matrix Profile I: All Pairs Similarity Joins for Time Series: A Unifying View That Includes Motifs, Discords and Shapelets. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, Piscataway, NJ, USA, 1317–1322. https://doi.org/10.1109/ICDM.2016.0179

[31] Byoung-Kee Yi, H. V. Jagadish, and Christos Faloutsos. 1998. Efficient Retrieval of Similar Time Sequences under Time Warping. In *Proceedings of the 14th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 201–208. https://doi.org/10.1109/ICDE.1998.655778

[32] Sheng Zhong and Abdullah Mueen. 2024. MASS: Distance Profile of a Query over a Time Series. *Data Mining and Knowledge Discovery* 38, 3 (2024), 1466–1492. https://doi.org/10.1007/s10618-024-01005-2